deduction. An agent's "program" – that is, its decision-making strategy – is encoded as a logical theory, and the process of selecting an action reduces to a problem of proof. Logic-based approaches are elegant, and have clean (logical) semantics – wherein lies much of their long-lived appeal. But logic-based approaches have many disadvantages. In particular, the inherent computational complexity of theorem proving makes it questionable whether agents as theorem provers can operate effectively in time-constrained environments. Decision-making in such agents is predicated on the assumption of calculative rationality – the assumption that the world will not change in any significant way while the agent is deciding what to do, and that an action which is rational when decision-making begins will be rational when it concludes. The problems associated with representing and reasoning about complex, dynamic, possibly physical environments are also essentially unsolved.

3.2 Agent-Oriented Programming

Yoav Shoham has proposed a 'new programming paradigm, based on a societal view of computation' which he calls *agent-oriented programming (AOP)*. The key idea which informs AOP is that of directly programming agents in terms of *mentalistic* notions (such as belief, desire, and intention) that agent theorists have developed to represent the properties of agents. The motivation behind the proposal is that humans use such concepts as an *abstraction* mechanism for representing the properties of complex systems. In the same way that we use these mentalistic notions to describe and explain the behaviour of humans, so it might be useful to use them to program machines. The idea of programming computer systems in terms of mental states was articulated in [Shoham, 1993].

AGENT-ORIENTED PROGRAMMING

The first implementation of the agent-oriented programming paradigm was the AGENT0 programming language. In this language, an agent is specified in terms of a set of *capabilities* (things the agent can do), a set of initial *beliefs*, a set of initial *commitments*, and a set of *commitment rules*. The key component, which determines how the agent acts, is the commitment rule set. Each commitment rule contains a *message condition*, a *mental condition*, and an action. In order to determine whether such a rule fires, the message condition is matched against the messages that the agent has received; the mental condition is matched against the beliefs of the agent. If the rule fires, then the agent becomes committed to the action.

AGENT0 LANGUAGE

COMMITMENT RULES

MESSAGE CONDITION

MENTAL CONDITION

Actions in AGENT0 may be *private*, corresponding to an internally executed subroutine, or *communicative*, i.e. sending messages. Messages are constrained to be one of three types: 'requests' or 'unrequests 'to perform or refrain from actions, and 'inform' messages, which pass on information (in Chapter 7 we will see that this style of communication is very common in multiagent systems). Request and unrequest messages typically result in the agent's commitments being modified; inform messages result in a change to the agent's beliefs.

PRIVATE ACTION

The operation of an AGENT0 agent can be described by the following loop:

1. Read all current messages, updating beliefs – and hence commitments – where necessary.
2. Execute all commitments for the current cycle where the capability condition of the associated action is satisfied.
3. Goto (1).

Figure 3.4 gives (part of) an AGENT0 agent definition. It should be clear how complex agent behaviours can be designed and built in AGENT0. However, it is important to note that this language is essentially a *prototype*, not intended for building anything like large-scale production systems. But it does at least give a feel for how such systems might be built.

3.3 Concurrent MetateM

The Concurrent MetateM language developed by Michael Fisher is based on the *direct execution* of logical formulae. In this sense, it comes very close to the 'ideal' of the agents as deductive theorem provers [Fisher, 1994]. A Concurrent MetateM system contains a number of concurrently executing agents, each of which is able to communicate with its peers via asynchronous broadcast message passing. Each agent is programmed by giving it a *temporal logic* specification of the behaviour that it is intended that the agent should exhibit. An agent's specification is executed directly to generate its behaviour. Execution of the agent program corresponds to iteratively building a logical model for the temporal agent specification. It is possible to prove that the procedure used to execute an agent specification is correct, in that if it is possible to satisfy the specification, then the agent will do so [Barringer et al., 1989].

CONCURRENT METATEM

TEMPORAL LOGIC

Agents in Concurrent MetateM are concurrently executing entities, able to communicate with each other through broadcast message passing. Each Concurrent MetateM agent has two main components:

- an *interface*, which defines how the agent may interact with its environment (i.e. other agents)
- a *computational engine*, which defines how the agent will act – in Concurrent MetateM, the approach used is based on the MetateM paradigm of executable temporal logic [Barringer et al., 1989].

Figure 3.4: Part of the definition of a 'plane' agent in AGENT0 [Torrance and Viola, 1991].

```
(defagent plane
  :timegrain 10
  :beliefs '(
       (1 (at 100 100))
       (1 (max-speed 5))
       (1 (CMT plane plane
```

```
(INFORM 2 world (2 (plane p1 100 100))))))

    :commit-rules
    '(
    ;; If I am requested to be at a certain place
    ;; at a certain time, then I do the private action
    ;; cap-check, to see if I am capable
    ;; of performing the requested action. If so, I
    ;; commit to perform the action. If not, nothing
    ;; happens.
    ( (control REQUEST (DO ?time (be-at ?gx ?gy)))
         () ;; no mental conditions
         control
         (DO now (cap-check ?time 'be-at ?gx ?gy)) )

    ;; If the control tower changes its mind, then I
    ;; uncommit to fly wherever it requested me to fly
    ( (control UNREQUEST (DO ?time (be-at ?gx ?gy)))
         (CMT control (DO ?time2 (be-at ?gx ?gy)))
         plane
         (DO now (uncommit ?time2 'be-at ?gx ?gy)) )

    ;; If I believe I am low on fuel and I believe I am committed to
    ;; control to fly to ?z1 ?z2 at time ?time2 then I want to ask
    ;; control to release me from my commitment to fly to (gx,gy).
    ( () ;; no message condition
         (and (B (now (lowfuel)))
              (CMT ?agent (DO ?time2 (be-at ?z1 ?z2))))
         plane
         (REQUEST now control (UNREQUEST now plane
              (DO ?time2 (be-at ?z1 ?z2)))) )
  [...]
    )          ; ends commitment rules
)       ; ends defagent
```

An agent interface consists of three components:

- a unique *agent identifier* (or just agent id), which names the agent
- a set of symbols defining which messages will be accepted by the agent – these are termed *environment propositions*
- a set of symbols defining messages that the agent may send – these are termed *component propositions*.

For example, the interface definition of a 'stack' agent might be

stack(*pop, push*) [*popped, full*].

Here, *stack* is the agent id that names the agent, {*pop, push*} is the set of environment propositions, and {*popped, full*} is the set of component propositions. The intuition is that, whenever a message headed by the symbol *pop* is broadcast, the *stack* agent will *accept* the message; we describe what this means below. If a message is broadcast that is not declared in the *stack* agent's interface, then *stack* ignores it. Similarly, the only messages that can be sent by the *stack* agent are headed by the symbols *popped* and *full*.

The computational engine of each agent in Concurrent MetateM is based on the MetateM paradigm of executable temporal logics [Barringer et al., 1989]. The idea is to directly execute

paradigm of executable temporal logics [Barringer et al., 1989]. The idea is to directly execute an agent specification, where this specification is given as a set of *program rules*, which are temporal logic formulae of the form:

PROGRAM RULES

antecedent about past → consequent about present and future.

The antecedent is a temporal logic formula referring to the past, whereas the consequent is a temporal logic formula referring to the present and future. The intuitive interpretation of such a rule is 'on the basis of the past, construct the future', which gives rise to the name of the paradigm: *declarative past and imperative future* [Gabbay, 1989]. The rules that define an agent's behaviour can be animated by directly executing the temporal specification under a suitable operational model [Fisher, 1995].

To make the discussion more concrete, we introduce a propositional temporal logic, called Propositional MetateM Logic (PML), in which the temporal rules that are used to specify an agent's behaviour will be given. (A complete definition of PML is given in [Barringer et al., 1989].) PML is essentially classical propositional logic augmented by a set of modal connectives for referring to the *temporal ordering* of events.

The meaning of the temporal connectives is quite straightforward: see Table 3.1 for a summary. Let φ and ψ be formulae of PML, then: $\bigcirc$ φ is satisfied at the current moment in time (i.e. now) if φ is satisfied at the next moment in time; $\Diamond$ φ is satisfied now if φ is satisfied either now or at some future moment in time; $\Box$ φ is satisfied now if φ is satisfied now and at all future moments; $\varphi\ U\ \psi$ is satisfied now if ψ is satisfied at some future moment, and φ is satisfied until then; and W is a binary connective similar to U, allowing for the possibility that the second argument might never be satisfied.

The past-time connectives have similar meanings: $\bullet$ φ is satisfied now if φ was satisfied at the previous moment in time; $\blacklozenge$ φ is satisfied now if φ was satisfied at some previous moment in time; $\blacksquare$ φ is satisfied now if φ was satisfied at all previous moments in time; $\varphi\ S\ \psi$ is satisfied now if ψ was satisfied at some previous moment in time, and φ has been satisfied since then; Z is similar, but allows for the possibility that the second argument was never satisfied; finally, a nullary temporal operator can be defined, which is satisfied only at the beginning of time – this useful operator is called 'start'.

To illustrate the use of these temporal connectives, consider the following examples:

$$\Box\ important(agents)$$

Table 3.1: Temporal connectives for Concurrent MetateM rules.

Operator	Meaning
$\bigcirc\ \phi$	ϕ is true 'tomorrow'
$\bullet\ \phi$	ϕ was true 'yesterday'
$\Diamond\ \phi$	at some time in the future, ϕ
$\Box\ \phi$	always in the future, ϕ
$\blacklozenge\ \phi$	at some time in the past, ϕ

◈ ϕ	at some time in the past, ϕ
■ ϕ	always in the past, ϕ
ϕ *U* ψ	ϕ will be true until ψ
ϕ *S* ψ	ϕ has been true since ψ
ϕ *W* ψ	ϕ is true unless ψ
ϕ *Z* ψ	ϕ is true since ψ

means 'it is now, and will always be true that agents are important'.

$$\Diamond \; important(Janine)$$

means 'sometime in the future, Janine will be important'.

$$(\neg friends(us))\ U\ apologize(you)$$

means 'we are not friends until you apologize'. And, finally,

$$\bigcirc \; apologize(you)$$

means 'tomorrow (in the next state), you apologize'.

The actual execution of an agent in Concurrent MetateM is, superficially at least, very simple to understand. Each agent obeys a cycle of trying to match the past-time antecedents of its rules against a *history*, and executing the consequents of those rules that 'fire'. More precisely, the computational engine for an agent continually executes the following cycle.

1. Update the *history* of the agent by receiving messages (i.e. environment propositions) from other agents and adding them to its history.
2. Check which rules *fire*, by comparing past-time antecedents of each rule against the current history to see which are satisfied.
3. *Jointly execute* the fired rules together with any commitments carried over from previous cycles.

   This involves first collecting together consequents of newly fired rules with old commitments – these become the *current constraints*. Now attempt to create the next state while satisfying these constraints. As the current constraints are represented by a disjunctive formula, the agent will have to choose between a number of execution possibilities.

   Note that it may not be possible to satisfy *all* the relevant commitments on the current cycle, in which case unsatisfied commitments are carried over to the next cycle.
4. Goto (1).

Clearly, step (3) is the heart of the execution process. Making the wrong choice at this step may mean that the agent specification cannot subsequently be satisfied.

When a proposition in an agent becomes *true*, it is compared against that agent's interface (see above); if it is one of the agent's *component propositions*, then that proposition is broadcast as a message to all other agents. On receipt of a message, each agent attempts to match the proposition against the environment propositions in its interface. If there is a match, then it adds the proposition to its history.

Figure 3.5: A simple Concurrent MetateM system.

$$
\begin{aligned}
&rp(ask1, ask2)[give1, give2]: \\
&\quad \bigcirc ask1 \rightarrow \Diamond give1; \\
&\quad \bigcirc ask2 \rightarrow \Diamond give2; \\
&\quad \text{start} \rightarrow \Box\neg(give1 \wedge give2).
\end{aligned}
$$

$$
\begin{aligned}
&rc1(give1)[ask1]: \\
&\quad \text{start} \rightarrow ask1; \\
&\quad \bigcirc ask1 \rightarrow ask1.
\end{aligned}
$$

$$
\begin{aligned}
&rc2(ask1, give2)[ask2]: \\
&\quad \bigcirc (ask1 \wedge \neg ask2) \rightarrow ask2.
\end{aligned}
$$

Figure 3.6: An example run of Concurrent MetateM.

	Agent		
Time	*rp*	*rc*1	*rc*2
0.		*ask*1	
1.	*ask*1	*ask*1	*ask*2
2.	*ask*1, *ask*2, *give*1	*ask*1	
3.	*ask*1, *give*2	*ask*1, *give*1	*ask*2
4.	*ask*1, *ask*2, *give*1	*ask*1	*give*2
5.	...	...	...

Figure 3.5 shows a simple system containing three agents: *rp, rc*1, and *rc*2. The agent *rp* is a 'resource producer': it can '*give*' to only one agent at a time, and will commit to eventually *give* to any agent that *ask*s. Agent *rp* will only accept messages *ask*1 and *ask*2, and can only send *give*1 and *give*2 messages. The interface of agent *rc*1 states that it will only accept *give*1 messages, and can only send *ask*1 messages. The rules for agent *rc*1 ensure that an *ask*1 message is sent on every cycle – this is because start is satisfied at the beginning of time, thus firing the first rule, so $\bigcirc$ *ask*1 will be satisfied on the next cycle, thus firing the second rule, and so on. Thus *rc*1 asks for the resource on every cycle, using an *ask*1 message. The interface for agent *rc*2 states that it will accept both *ask*1 and *give*2 messages, and can send *ask*2 messages. The single rule for agent *rc*2 ensures that an *ask*2 message is sent on every cycle where, on its previous cycle, it did not send an *ask*2 message, but received an *ask*1 message (from agent *rc*1). Figure 3.6 shows a fragment of an example run of the system in Figure 3.5.

Notes and Further Reading

My presentation of logic-based agents draws heavily on the discussion of *deliberate agents* presented in [Genesereth and Nilsson, 1987, Chapter 13], which represents the logic-centric view of AI and agents very well. The discussion is also partly based on [Konolige, 1986]. A number of more-or-less 'pure' logical approaches to agent programming have been developed. Well-known examples include the ConGolog system of Lespérance and colleagues [Lésperance et al., 1996] (which is based on the *situation calculus* [McCarthy and Hayes, 1969]). Note that these architectures (and the discussion above) assume that if one adopts a logical approach to agent building, then this means agents are essentially theorem provers, employing explicit symbolic reasoning (theorem proving) in order to make decisions. But just because we find logic a useful tool for conceptualizing or specifying agents, this does not mean that we must view decision-making as logical manipulation. An alternative is to *compile* the logical specification of an agent into a form more amenable to efficient decision-making. The difference is rather like the distinction between interpreted and compiled programming languages. The best-known example of this work is the *situated*

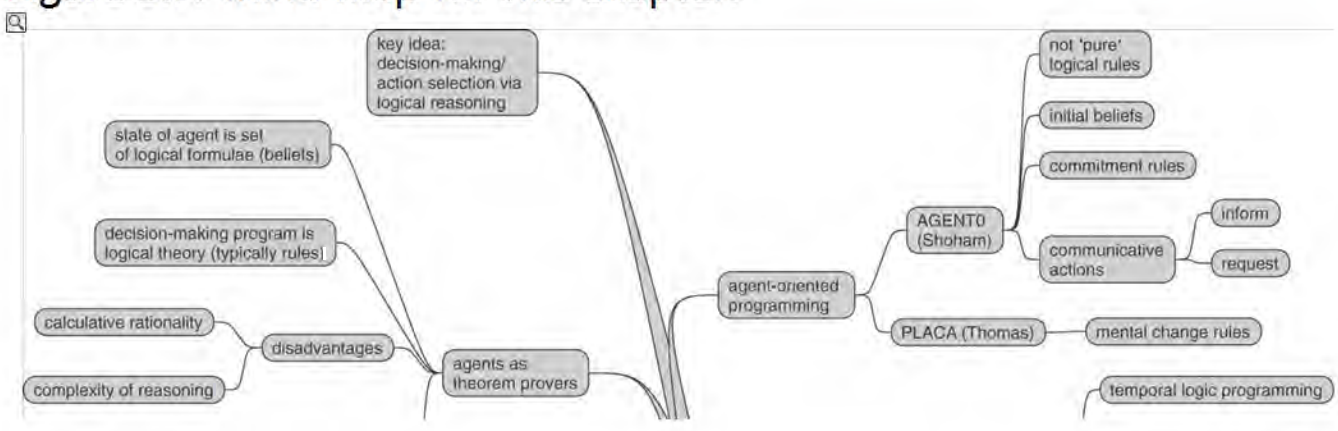
automata paradigm of [Rosenschein and Kaelbling, 1996]. A review of the role of logic in intelligent agents may be found in [Wooldridge, 1997]. Finally, for a detailed discussion of calculative rationality and the way that it has affected thinking in AI see [Russell and Subramanian, 1995].

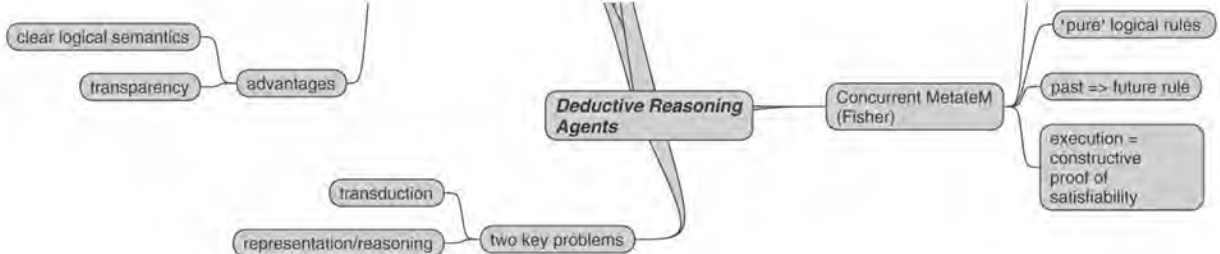
The main references to AGENT0 are [Shoham, 1990, 1993]. Shoham's AOP proposal has been enormously influential in the multiagent systems community. In addition to the reasons set out in the main text, there are other reasons for believing that an intentional stance will be useful for understanding and reasoning about computer programs [Huhns and Singh, 1998]. First, and perhaps most importantly, the ability of heterogeneous, self-interested agents to communicate seems to imply the ability to talk about the beliefs, aspirations, and intentions of individual agents. For example, in order to *coordinate* their activities, agents must have information about the intentions of others [Jennings, 1993a]. This idea is closely related to Newell's *knowledge level* [Newell, 1982]. Later in this book, we will see how mental states such as beliefs, desires, and the like are used to give a semantics to *speech acts* [Cohen and Levesque, 1990a; Searle, 1969]. Second, mentalistic models are a good candidate for representing information about end users. For example, imagine a tutoring system that works with students to teach them Java programming. One way to build such a system is to give it a *model* of the user. Beliefs, desires, intentions, and the like seem appropriate for the make-up of such models.

Michael Fisher's Concurrent MetateM language is described in [Fisher, 1994]; the execution algorithm that underpins it is described in [Barringer et al., 1989]. Since Shoham's proposal, a number of languages have been proposed which claim to be agent oriented. Examples include Becky Thomas's Planning Communicating Agents (PLACA) language [Thomas, 1993, 1995], and the AgentSpeak(L) language [Bordini et al., 2007; Rao, 1996a]. A representative collection of papers on agent programming languages is [Bordini et al., 2005].

Class reading: [Shoham, 1993]. This is the article that introduced agent-oriented programming and, throughout the late 1990s, was one of the most cited articles in the agent community. The main point about the article, as far as I am concerned, is that it explicitly articulates the idea of programming systems in terms of 'mental states'. AGENT0, the actual language described in the article, is not a language that you would be likely to use for developing 'real' systems. A useful discussion might be had on (i) whether 'mental states' are really useful in programming systems; (ii) how one might go about proving or disproving the hypothesis that mental states are useful in programming systems; and (iii) how AGENT0-like features might be incorporated in a language such as Java.

Figure 3.7: Mind map for this chapter.




clear logical semantics
transparency
advantages
Deductive Reasoning Agents
Concurrent MetateM (Fisher)
'pure' logical rules
past => future rule
execution = constructive proof of satisfiability
transduction
representation/reasoning
two key problems

Chapter 4 Practical Reasoning Agents

Whatever the merits of agents that decide what to do by proving theorems, it seems clear that *we* do not use purely logical reasoning in order to decide what to do. Certainly something like logical reasoning can play a part, but a moment's reflection should confirm that for most of the time, very different processes are taking place. In this chapter, I will focus on a model of agency that takes its inspiration from the processes that seem to take place as we decide what to do.

4.1 Practical Reasoning = Deliberation + Means–Ends Reasoning

The particular model of decision-making is known as *practical reasoning*. Practical reasoning is reasoning directed towards actions – the process of figuring out what to do.

PRACTICAL REASONING

> Practical reasoning is a matter of weighing conflicting considerations for and against competing options, where the relevant considerations are provided by what the agent desires/values/cares about and what the agent believes.
>
> [Bratman, 1990, p. 17]

It is important to distinguish practical reasoning from *theoretical reasoning* [Eliasmith, 1999]. Theoretical reasoning is directed towards beliefs. To use a rather tired example, if I believe that all men are mortal, and I believe that Socrates is a man, then I will usually conclude that Socrates is mortal. The process of concluding that Socrates is mortal is theoretical reasoning, since it affects only my beliefs about the world. The process of deciding to catch a bus instead of a train, however, is practical reasoning, since it is reasoning directed towards action.

THEORETICAL REASONING

Human practical reasoning appears to consist of at least two distinct activities. The first of these involves deciding *what* state of affairs we want to achieve; the second process involves deciding *how* we want to achieve these states of affairs. The former process – deciding what states of affairs to achieve – is known as *deliberation*. The latter process – deciding how to achieve these states of affairs – we call *means–ends reasoning*.

DELIBERATION

MEANS-ENDS REASONING

To better understand deliberation and means–ends reasoning, consider the following example. When a person graduates from university with a first degree, he or she is faced with some important choices. Typically, one proceeds in these choices by first deciding what sort of career to follow. For example, one might consider a career as an academic, or a career in industry. The process of deciding which career to aim for is deliberation. Once one has fixed upon a career, there are further choices to be made; in particular, how to bring about this career. Suppose that, after deliberation, you choose to pursue a career as an academic. The next step is to decide *how to achieve* this state of affairs. This process is means–ends reasoning. The end result of means–ends reasoning is a *plan* or *recipe* of some kind for achieving the chosen state of affairs. For the career example, a plan might involve first applying to an appropriate university for a

PhD place, and so on. After obtaining a plan, an agent will typically then attempt to carry out (or *execute*) the plan, in order to bring about the chosen state of affairs. If all goes well (the plan is sound, and the agent's environment cooperates sufficiently), then after the plan has been executed, the chosen state of affairs will be achieved.

PLANS/RECIPES

Thus described, practical reasoning seems a straightforward process, and in an ideal world, it would be. But there are several complications. The first is that deliberation and means–ends reasoning are *computational* processes. In all real agents (and, in particular, artificial agents), such computational processes will take place under *resource bounds*. By this I mean that an agent will only have a fixed amount of memory and a fixed processor available to carry out its computations. Together, these resource bounds impose a limit on the size of computations that can be carried out in any given amount of time. No real agent will be able to carry out arbitrarily large computations in a finite amount of time. Since almost any real environment will also operate in the presence of *time constraints* of some kind, this means that means–ends reasoning and deliberation must be carried out in a fixed, finite number of processor cycles, with a fixed, finite amount of memory space. From this discussion, we can see that resource bounds have two important implications:

- Computation is a valuable resource for agents situated in real-time environments. The ability to perform well will be determined at least in part by the ability to make efficient use of available computational resources. In other words, an agent must *control* its reasoning effectively if it is to perform well.
- Agents cannot deliberate indefinitely. They must clearly *stop* deliberating at some point, having chosen some state of affairs, and commit to achieving this state of affairs. It may well be that the state of affairs an agent has fixed upon is not optimal – further deliberation might have led it to fix upon another state of affairs.

We refer to the states of affairs that an agent has chosen and committed to as its *intentions*.

INTENTIONS

Intentions in practical reasoning

First, notice that it is possible to distinguish several different types of intention. In ordinary speech, we use the term 'intention' to characterize both *actions* and *states of mind*. To adapt an example from Bratman [Bratman, 1987, p. 1], I might intentionally push someone under a train, and push them with the intention of killing them. Intention is here used to characterize an action – the action of pushing someone under a train. Alternatively, I might have the intention this morning of pushing someone under a train this afternoon. Here, intention is used to characterize my state of mind. In this book, when I talk about intentions, I mean intentions as states of mind. In particular, I mean *future-directed intentions* – intentions that an agent has towards some future state of affairs.

FUTURE-DIRECTED INTENTIONS

The most obvious role of intentions is that they are *pro-attitudes* [Bratman, 1990, p. 23]. By this, I mean that they tend to lead to action. Suppose I have an intention to write a book. If I truly have such an intention, then you would expect me to make a *reasonable attempt* to

truly have such an intention, then you would expect me to make a *reasonable attempt* to achieve it. This would usually involve, at the very least, me initiating some plan of action that I believed would satisfy the intention. In this sense, intentions tend to play a primary role in the production of action. As time passes, and my intention about the future becomes my intention about the present, then it plays a direct role in the production of action. Of course, having an intention does not necessarily lead to action. For example, I can have an intention now to attend a conference later in the year. I can be utterly sincere in this intention, and yet if I learn of some event that must take precedence over the conference, I may never even get as far as considering travel arrangements.

PRO-ATTITUDES

Bratman notes that intentions play a much stronger role in influencing action than other pro-attitudes, such as mere desires.

> My desire to play basketball this afternoon is merely a potential influencer of my conduct this afternoon. It must vie with my other relevant desires … before it is settled what I will do. In contrast, once I intend to play basketball this afternoon, the matter is settled: I normally need not continue to weigh the pros and cons. When the afternoon arrives, I will normally just proceed to execute my intentions.
>
> [Bratman, 1990, p. 22]

The second main property of intentions is that they *persist*. If I adopt an intention to become an academic, then I should persist with this intention and attempt to achieve it. For if I immediately drop my intentions without devoting any resources to achieving them, then I will not be acting rationally. Indeed, you might be inclined to say that I never really had intentions in the first place.

Of course, I should not persist with my intention for too long – if it becomes clear to me that I will never become an academic, then it is only rational to drop my intention to do so. Similarly, if the reason for having an intention goes away, then it would be rational for me to drop the intention. For example, if I adopted the intention to become an academic because I believed it would be an easy life, but then discover that this is not the case (e.g. I might be expected to actually teach!), then the justification for the intention is no longer present, and I should drop the intention.

If I initially fail to achieve an intention, then you would expect me to *try again* – you would not expect me to simply give up. For example, if my first application for a PhD programme is rejected, then you might expect me to apply to alternative universities.

The third main property of intentions is that once I have adopted an intention, the very fact of having this intention will constrain my future practical reasoning. For example, while I hold some particular intention, I will not subsequently entertain options that are *inconsistent* with that intention. Intending to write a book, for example, would preclude the option of partying every night: the two are mutually exclusive. This is in fact a highly desirable property from the point of view of implementing rational agents, because in providing a 'filter of admissibility', intentions can be seen to constrain the space of possible intentions that an agent needs to consider.

Finally, intentions are closely related to beliefs about the future. For example, if I intend to become an academic, then I should believe that, assuming certain background conditions are

satisfied, I will indeed become an academic. For if I truly believe that I will never be an academic, it would be nonsensical of me to have an intention to become one. Thus if I intend to become an academic, I should at least believe that there is a good chance I will indeed become one. However, there is what appears at first sight to be a paradox here. While I might believe that I will indeed succeed in achieving my intention, if I am rational, then I must also recognize the possibility that I can *fail* to bring it about – that there is some circumstance under which my intention is not satisfied.

From this discussion, we can identify the following closely related situations.

- Having an intention to bring about φ, while believing that you will not bring about φ, is called *intention–belief inconsistency*, and is not rational (see, for example, [Bratman, 1987]).

INTENTION-BELIEF INCONSISTENCY

- Having an intention to achieve φ without believing that φ will be the case is *intention–belief incompleteness*, and is an acceptable property of rational agents (see, for example, [Bratman, 1987, p. 38]).

The distinction between these two cases is known as the *asymmetry thesis* [Bratman, 1987, pp. 37–41].

ASYMMETRY THESIS

Summarizing, we can see that intentions play the following important roles in practical reasoning.

Intentions drive means–ends reasoning If I have formed an intention, then I will attempt to achieve the intention, which involves, among other things, deciding *how* to achieve it. Moreover, if one particular course of action fails to achieve an intention, then I will typically attempt others.

Intentions persist I will not usually give up on my intentions without good reason – they will persist, typically until I believe I have successfully achieved them, I believe I cannot achieve them, or I believe the reason for the intention is no longer present.

Intentions constrain future deliberation I will not entertain options that are inconsistent with my current intentions.

Intentions influence beliefs upon which future practical reasoning is based If I adopt an intention, then I can plan for the future on the assumption that I will achieve the intention. For if I intend to achieve some state of affairs while simultaneously believing that I will not achieve it, then I am being irrational.

Notice from this discussion that intentions *interact* with an agent's beliefs and other mental states. For example, having an intention to achieve φ implies that I do not believe ϕ is impossible, and moreover that I believe that, given the right circumstances, ϕ will be achieved. However, satisfactorily capturing the interaction between intention and belief turns out to be surprisingly hard – some discussion on this topic appears in Chapter 17.

Throughout the remainder of this chapter, I make one important assumption: that the agent

maintains some explicit *representation* of its beliefs, desires, and intentions. However, I will not be concerned with *how* beliefs and the like are represented. One possibility is that they are represented *symbolically*, for example as logical statements *á la* Prolog facts [Clocksin and Mellish, 1981]. However, the assumption that beliefs, desires, and intentions are symbolically represented is by no means necessary for the remainder of the book. I use B to denote a variable that holds the agent's current beliefs, and let Bel be the set of all such beliefs. Similarly, I use D as a variable for desires, and Des to denote the set of all desires. Finally, the variable I represents the agent's intentions, and Int is the set of all possible intentions.

In what follows, deliberation will be modelled via two functions:

- an option generation function
- a filtering function.

The signature of the option generation function *options* is as follows:

$$options : 2^{Bel} \times 2^{Int} \rightarrow 2^{Des}.$$

This function takes the agent's current beliefs and current intentions, and on the basis of these produces a set of possible options or desires.

In order to select between competing options, an agent uses a *filter* function. Intuitively, the filter function must simply select the 'best' option(s) for the agent to commit to. We represent the filter process through a function *filter*, with a signature as follows:

DESIRE FILTERING

$$filter : 2^{Bel} \times 2^{Des} \times 2^{Int} \rightarrow 2^{Int}.$$

An agent's belief update process is modelled through a *belief revision function*:

$$brf : 2^{Bel} \times \text{Per} \rightarrow 2^{Bel}.$$

4.2 Means–Ends Reasoning

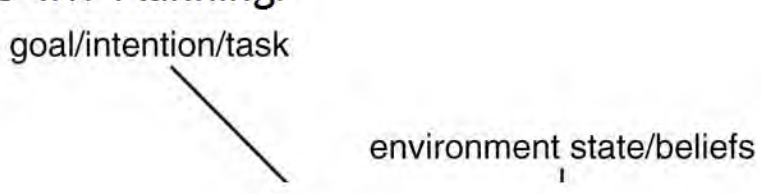
Means–ends reasoning is the process of deciding how to achieve an end (i.e. an intention that you have) using the available means (i.e. the actions that you can perform). Means–ends reasoning is perhaps better known in the AI community as *planning*.

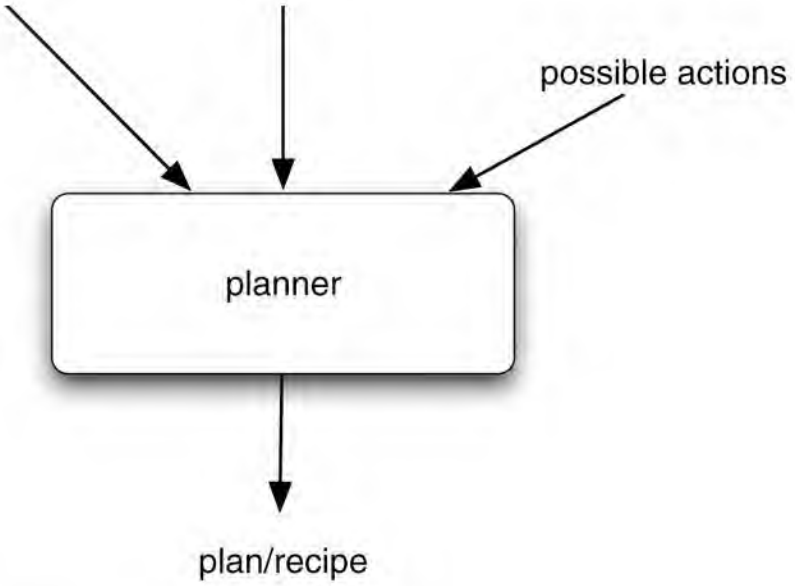
PLANNING

Planning is essentially automatic programming. A planner is a system that takes as input representations of the following.

1. a *goal, intention* or (in the terminology of Chapter 2) a *task*. This is something that the agent wants to achieve (in the case of achievement tasks – see Chapter 2), or a state of affairs that the agent wants to maintain or avoid (in the case of maintenance tasks – see Chapter 2)

Figure 4.1: Planning.





2. the current *state of the environment* – the agent's *beliefs*
3. the *actions* available to the agent.

As output, a planning algorithm generates a *plan* (see Figure 4.1). This is a course of action – a 'recipe'. If the planning algorithm does its job correctly, then if the agent executes this plan ('follows the recipe') from a state in which the world is as described in (2), then once the plan has been completely executed, the goal/intention/task described in (1) will be carried out.

The first real planner was the STRIPS system, developed by Fikes in the late 1960s/early 1970s [Fikes and Nilsson, 1971]. The two basic components of STRIPS were a model of the world as a set of formulae of first-order logic, and a set of *action schemata*, which describe the preconditions and effects of all the actions available to the planning agent. This latter component has perhaps proved to be STRIPS's most lasting legacy in the AI planning community: nearly all implemented planners employ the 'STRIPS formalism' for action, or some variant of it. The STRIPS planning algorithm was based on a principle of finding the 'difference' between the current state of the world and the goal state, and reducing this difference by applying an action. Unfortunately, this proved to be an inefficient process for formulating plans, as STRIPS tended to become 'lost' in low-level plan detail.

STRIPS NOTATION

There is not scope in this book to give a detailed technical introduction to planning algorithms and technologies, and in fact it is probably not appropriate to do so. Nevertheless, it is at least worth giving a short overview of the main concepts.

Figure 4.2: The Blocks World.

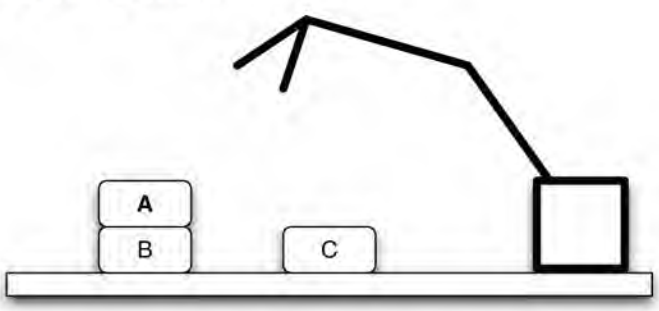


Table 4.1: Predicates for describing the Blocks World.

Predicate	Meaning
On (*x*, *y*)	object *x* is on top of object *y*
OnTable(*x*)	object *x* is on the table
Clear(*x*)	nothing is on top of object *x*
Holding(*x*)	robot arm is holding *x*
Arm Empty	robot arm is empty (not holding anything)

The Blocks World

In time-honoured fashion, I will illustrate the techniques with reference to a *Blocks World*. The Blocks World contains three blocks (*A*, *B*, and *C*) of equal size, a robot arm capable of picking up and moving one block at a time, and a table top. The blocks may be placed on the table top, or may be placed one on top of another. Figure 4.2 shows one possible configuration of the Blocks World.

Notice that, in the description of planning algorithms I gave above, I stated that planning algorithms take as input *representations* of the goal, the current state of the environment, and the actions available. The first issue is exactly what form these representations take. The STRIPS system made use of representations based on first-order logic. I will use the predicates in Table 4.1 to represent the Blocks World.

A description of the Blocks World in Figure 4.2 is, using these predicates, as follows:

$$\{Clear(A), On(A, B), OnTable(B), OnTable(C), Clear(C)\}.$$

I am implicitly making use of the *closed world* assumption: if something is not explicitly stated to be true, then it is assumed false.

The next issue is how to represent goals. Again, we represent a goal as a set of formulae of first-order logic:

$$\{OnTable(A), OnTable(B), OnTable(C)\}.$$

So the goal is that all the blocks are on the table. To represent actions, we make use of the precondition/delete/add list notation – the STRIPS formalism. In this formalism, each action has

- a *name* – which may have arguments
- a *precondition list* – a list of facts which must be true for the action to be executed

  PRECONDITION LIST
- a *delete list* – a list of facts that are no longer true after the action is performed

  DELETE LIST
- an *add list* – a list of facts made true by executing the action.

  ADD LIST

The *Stack* action occurs when the robot arm places the object *x* that it is holding on top of object *y*:

$$Stack(x, y)$$

pre {*Clear*(*y*), *Holding*(*x*)}

del {*Clear*(*y*), *Holding*(*x*)}

add {*ArmEmpty*, *On*(*x*, *y*)}

The *UnStack* action occurs when the robot arm picks an object *x* up from on top of another object *y*:

UnStack(*x*, *y*)

pre {*On*(*x*, *y*), *Clear*(*x*), *ArmEmpty*}

del {*On*(*x*, *y*), *ArmEmpty*}

add {*Holding*(*x*), *Clear*(*y*)}

The *Pickup* action occurs when the arm picks up an object *x* from the table:

Pickup(*x*)

pre {*Clear*(*x*), *OnTable*(*x*), *ArmEmpty*}

del {*OnTable*(*x*), *ArmEmpty*}

add {*Holding*(*x*)}

The *PutDown* action occurs when the arm places the object *x* onto the table:

PutDown(*x*)

pre {*Holding*(*x*)}

del {*Holding*(*x*)}

add {*ArmEmpty*, *OnTable*(*x*)}

Let us now describe what is going on somewhat more formally. First, as we have throughout the book, we assume a fixed set of actions $Ac = \{\alpha_1, \ldots, \alpha_n\}$ that the agent can perform. A *descriptor* for an action $\alpha \in Ac$ a triple

$$(P_\alpha, D_\alpha, A_\alpha),$$

where

- P_α is a set of formulae of first-order logic that characterize the *precondition* of action α
- D_α is a set of formulae of first-order logic that characterize those facts made *false* by the performance of α (the *delete list*)
- A_α is a set of formulae of first-order logic that characterize those facts made *true* by the performance of α (the *add list*).

For simplicity, we will assume that the precondition, delete, and add lists are constrained to only contain *ground atoms* – individual predicates, which do not contain logical connectives or variables.

A *planning problem* (over the set of actions *Ac*) is then determined by a triple

$$(\Delta, o, \gamma),$$

where

- Δ is the beliefs of the agent about the *initial* state of the world – these beliefs will be a set

of formulae of first order (cf. the vacuum world in Chapter 2)

- $o = \{(P_\alpha, D_\alpha, A_\alpha) \mid \alpha \in Ac\}$ is an indexed set of operator descriptors, one for each available action α
- γ is a set of formulae of first-order logic, representing the goal/task/intention to be achieved.

A *plan* π is a sequence of actions

$$\pi = (\alpha_1, \ldots, \alpha_n),$$

where each α_i is a member of Ac.

With respect to a planning problem (Δ, o, γ), a plan $\pi = (\alpha_1, \ldots, \alpha_n)$ determines a sequence of $n + 1$ belief databases

$$\Delta_0, \Delta_1, \ldots, \Delta_n,$$

where

$$\Delta_0 = \Delta$$

and

$$\Delta_i = (\Delta_{i-1} \setminus D\alpha_i) \cup A\alpha_i \text{ for } 1 \leq i \leq n.$$

A (linear) plan $\pi = (\alpha_1, \ldots, \alpha_n)$ is said to be *acceptable* with respect to the problem (Δ, o, γ) if, and only if, the precondition of every action is satisfied in the preceding belief database, i.e. if $\Delta_{i-1} \models P\alpha_i$, for all $1 \leq i \leq$ n. A plan $\pi = (\alpha_1, \ldots, \alpha_n)$ is *correct* with respect to (Δ, o, γ) if and only if

ACCEPTABLE PLAN

CORRECT PLAN

1. it is acceptable and
2. $\Delta_n \models \gamma$ (i.e. if the goal is achieved in the final belief database generated by the plan).

The problem to be solved by a planning system can then be stated as follows.

Given a planning problem (Δ, o, γ), find a correct plan for (Δ, o, γ) or else announce that none exists.

(It is worth comparing this discussion with that on the synthesis of agents in Chapter 3 – similar comments apply with respect to the issues of soundness and completeness.)

We will use π (with decorations: π', π_1, ...) to denote plans, and let *Plan* be the set of all plans (over some set of actions Ac). We will make use of a number of auxiliary definitions for manipulating plans (some of these will not actually be required until later in this chapter):

- if π is a plan, then we write $pre(\pi)$ to denote the precondition of π, and $body(\pi)$ to denote the body of π
- if π is a plan, then we write $empty(\pi)$ to mean that plan π is the empty sequence (thus

empty(…) is a Boolean-valued function)

- *execute*(…) is a procedure that takes as input a single plan and executes it without stopping – executing a plan simply means executing each action in the plan body in turn
- if π is a plan, then by *hd*(π) we mean the plan made up of the first action in the plan body of π; for example, if the body of π is $\alpha_1, \ldots, \alpha_n$, then the body of *hd*($\pi$) contains only the action α_1
- if π is a plan, then by *tail*(π) we mean the plan made up of all but the first action in the plan body of π; for example, if the body of π is $\alpha_1, \alpha_2, \ldots, \alpha_n$, then the body of *tail*(π) contains actions $\alpha_2, \ldots, \alpha_n$
- if π is a plan, $I \subseteq Int$ is a set of intentions, and $B \subseteq Bel$ is a set of beliefs, then we write *sound*(π, *I*, *B*) to mean that π is a correct plan for intentions *I* given beliefs *B* [Lifschitz, 1986].

An agent's means–ends reasoning capability is represented by a function

$$plan : \mathbf{2}^{Bel} \times \mathbf{2}^{Int} \times \mathbf{2}^{Ac} \rightarrow Plan,$$

which, on the basis of an agent's current beliefs and current intentions, determines a plan to achieve the intentions.

PLAN LIBRARY

Notice that there is nothing in the definition of the *plan*(…) function which requires an agent to engage in *plan generation* – constructing a plan from scratch [Allen et al., 1990]. In many implemented practical reasoning agents, the *plan*(…) function is implemented by giving the agent a *plan library* [Georgeff and Lansky, 1987]. A plan library is a pre-assembled collection of plans, which an agent designer gives to an agent. Finding a plan to achieve an intention then simply involves a single pass through the plan library to find a plan that, when executed, will have the intention as a postcondition, and will be sound given the agent's current beliefs. Preconditions and postconditions for plans are often represented as (lists of) atoms of first-order logic, and beliefs and intentions as ground atoms of first-order logic. Finding a plan to achieve an intention then reduces to finding a plan whose precondition unifies with the agent's beliefs, and whose postcondition unifies with the intention. At the end of this chapter, we will see how this idea works in the Procedural Reasoning System (PRS) system.

Figure 4.3: A practical reasoning agent.

```
1.  B ← B0;          /* B0 are initial beliefs */
2.  I ← I0;          /* I0 are initial intentions */
3.  while true do
4.      get next percept ρ through see(...) function;
5.      B ← brf(B, ρ);
6.      D ← options(B, I);
7.      I ← filter(B, D, I);
8.      π ← plan(B, I, Ac);
9.      while not (empty(π) or succeeded(I, B) or impossible(I, B)) do
10.         α ← hd(π);
11.         execute(α);
12.         π ← tail(π);
13.         get next percept ρ through see(...) function;
```

```
13.          get next percept ρ through see(...) function;
14.          B ← brf(B, ρ);
15.          if reconsider(I, B) then
16.              D ← options(B, I);
17.              I ← filter(B, D, I);
18.          end-if
19.          if not sound(π, I, B) then
20.              π ← plan(B, I, Ac);
21.          end-if
22.      end-while
23. end-while
```

4.3 Implementing a Practical Reasoning Agent

We can now discuss the overall control structure of a practical reasoning agent. Figure 4.3 gives the pseudo-code for the control cycle of such an agent. The basic structure of the decision-making process is a loop, in which the agent continually

- observes the world, and updates beliefs
- deliberates to decide what intention to achieve (deliberation being done by first determining the available options and then by filtering)
- uses means–ends reasoning to find a plan to achieve these intentions
- executes the plan.

However, this basic control loop is complicated by a number of concerns. The first of these is that of *commitment* – and, in particular, how committed an agent is to both ends (the intention) and means (the plan to achieve the intention).

COMMITMENTS

Commitments to ends and means

When an option successfully passes through the *filter* function and is hence chosen by the agent as an intention, we say that the agent has made a *commitment* to that option. Commitment implies *temporal persistence* – an intention, once adopted, should not immediately evaporate. A critical issue is just *how* committed an agent should be to its intentions. That is, how long should an intention persist? Under what circumstances should an intention vanish?

To motivate the discussion further, consider the following scenario.

Some time in the not-so-distant future, you are having trouble with your new household robot. You say "Willie, bring me a beer." The robot replies "OK boss." Twenty minutes later, you screech "Willie, why didn't you bring me that beer?" It answers "Well, I intended to get you the beer, but I decided to do something else." Miffed, you send the wise guy back to the manufacturer, complaining about a lack of commitment. After retrofitting, Willie is returned, marked "Model C: The Committed Assistant." Again, you ask Willie to bring you a beer. Again, it accedes, replying "Sure thing." Then you ask: "What kind of beer did you buy?" It answers: "Genessee." You say "Never mind." One minute later, Willie trundles over with a Genessee in its gripper. This time, you angrily return Willie for overcommitment. After still more tinkering, the manufacturer sends Willie back, promising no more problems with its commitments. So, being a somewhat trusting customer, you

no more problems with its commitments. So, being a somewhat trusting customer, you accept the rascal back into your household, but as a test, you ask it to bring you your last beer. Willie again accedes, saying "Yes, Sir." (Its attitude problem seems to have been fixed.) The robot gets the beer and starts towards you. As it approaches, it lifts its arm, wheels around, deliberately smashes the bottle, and trundles off. Back at the plant, when interrogated by customer service as to why it had abandoned its commitments, the robot replies that according to its specifications, it kept its commitments as long as required – commitments must be dropped when fulfilled or impossible to achieve. By smashing the bottle, the commitment became unachievable.

[Cohen and Levesque, 1990a, pp. 213, 214]

The mechanism that an agent uses to determine when and how to drop intentions is known as a *commitment strategy*. The following three commitment strategies are commonly discussed in the literature of rational agents [Rao and Georgeff, 1991b].

COMMITMENT STRATEGY

Blind commitment A blindly committed agent will continue to maintain an intention until it believes that the intention has actually been achieved. Blind commitment is also sometimes referred to as *fanatical* commitment.

BLIND COMMITMENT

Single-minded commitment A single-minded agent will continue to maintain an intention until it believes either that the intention has been achieved, or else that it is no longer possible to achieve the intention.

SINGLE-MINDED COMMITMENT

Open-minded commitment An open-minded agent will maintain an intention as long as it is still believed possible.

OPEN-MINDED COMMITMENT

Note that an agent has commitment both to *ends* (i.e. the state of affairs it wishes to bring about) and *means* (i.e. the mechanism via which the agent wishes to achieve the state of affairs).

With respect to commitment to means (i.e. plans), the solution adopted in Figure 4.3 is as follows. An agent will maintain a commitment to an intention until (i) it believes the intention has succeeded; (ii) it believes the intention is impossible; or (iii) there is nothing left to execute in the plan. This is single-minded commitment. We write *succeeded*(*I, B*) to mean that given beliefs *B*, the intentions *I* can be regarded as having been satisfied. Similarly, we write *impossible*(*I, B*) to mean that intentions *I* are impossible given beliefs *B*. The main loop, capturing this commitment to means, is in lines 9–22.

How about commitment to ends? When should an agent stop to *reconsider* its intentions? One possibility is to reconsider intentions at every opportunity – in particular, after executing every possible action. If option generation and filtering were computationally cheap processes, then this would be an acceptable strategy. Unfortunately, we know that deliberation is not cheap it

this would be an acceptable strategy. Unfortunately, we know that deliberation is not cheap it takes a considerable amount of time. While the agent is deliberating, the environment in which the agent is working is changing, possibly rendering its newly formed intentions irrelevant.

INTENTION RECONSIDERATION

We are thus presented with a dilemma:

- An agent that does not stop to reconsider its intentions sufficiently often will continue attempting to achieve its intentions even after it is clear that they cannot be achieved, or that there is no longer any reason for achieving them.
- An agent that *constantly* reconsiders its attentions may spend insufficient time actually working to achieve them, and hence runs the risk of never actually achieving them.

There is clearly a trade-off to be struck between the degree of commitment and reconsideration at work here. To try to capture this trade-off, Figure 4.3 incorporates an explicit *meta-level control* component. The idea is to have a Boolean-valued function, *reconsider*, such that *reconsider*(*I, B*) evaluates to 'true' just in case it is appropriate for the agent with beliefs *B* and intentions *I* to reconsider its intentions. Deciding whether to reconsider intentions thus falls to this function.

It is interesting to consider the circumstances under which this function can be said to behave *optimally*. Suppose that the agent's deliberation and plan generation functions are in some sense perfect: that deliberation always chooses the 'best' intentions (however that is defined for the application at hand), and planning always produces an appropriate plan. Further suppose that time expended always has a cost – the agent does not benefit by doing nothing. Then it is not difficult to see that the function *reconsider*(…) will be behaving optimally if, and only if, whenever it chooses to deliberate, the agent changes intentions [Wooldridge and Parsons, 1999]. For if the agent chose to deliberate but did not change intentions, then the effort expended on deliberation was wasted. Similarly, if an agent should have changed intentions, but failed to do so, then the effort expended on attempting to achieve its intentions was also wasted.

Table 4.2: Practical reasoning situations (cf. [Bratman et al., 1988, p. 353]).

Situation number	Chose to deliberate?	Changed intentions?	Would have changed intentions?	*reconsider(...)* optimal?
1.	No	—	No	Yes
2.	No	—	Yes	No
3.	Yes	No	—	No
4.	Yes	Yes	—	Yes

The possible interactions between deliberation and meta-level control (the function *reconsider*(…)) are summarized in Table 4.2.

- In situation (1), the agent did not choose to deliberate, and, as a consequence, did not choose to change intentions. Moreover, if it *had* chosen to deliberate, it would not have changed intentions. In this situation, the *reconsider*(…) function is behaving optimally.
- In situation (2), the agent did not choose to deliberate, but if it had done so it *would* have

changed intentions. In this situation, the *reconsider*(…) function is not behaving optimally.

- In situation (3), the agent chose to deliberate, but did not change intentions. In this situation, the *reconsider*(…) function is not behaving optimally.
- In situation (4), the agent chose to deliberate, and did change intentions. In this situation, the *reconsider*(…) function is behaving optimally.

Notice that there is an important assumption implicit within this discussion: that the cost of executing the *reconsider*(…) function is *much* less than the cost of the deliberation process itself. Otherwise, the *reconsider*(…) function could simply use the deliberation process as an oracle, running it as a subroutine and choosing to deliberate just in case the deliberation process changed intentions.

The nature of the trade-off was examined by David Kinny and Michael Georgeff in a number of experiments carried out using a BDI (belief–desire–intention) agent system [Kinny and Georgeff, 1991]. The aims of Kinny and Georgeff's investigation were to

> (1) assess the feasibility of experimentally measuring agent effectiveness in a simulated environment, (2) investigate how commitment to goals contributes to effective agent behaviour and (3) compare the properties of different strategies for reacting to change.
>
> [Kinny and Georgeff, 1991, p. 82]

In Kinny and Georgeff's experiments, two different types of reconsideration strategy were used: *bold* agents, which never pause to reconsider their intentions before their current planis fully executed, and *cautious* agents, which stop to reconsider after the execution of every action. These characteristics are defined by a *degree of boldness*, which specifies the maximum number of plan steps the agent executes before reconsidering its intentions. Dynamism in the environment is represented by the *rate of environment change*. Put simply, the rate of environment change is the ratio of the speed of the agent's control loop to the rate of change of the environment. If the rate of world change is 1, then the environment will change no more than once for each time the agent can execute its control loop. If the rate of world change is 2, then the environment can change twice for each pass through the agent's control loop, and so on. The performance of an agent is measured by the ratio of the number of intentions that the agent managed to achieve to the number of intentions that the agent had at any time. Thus if effectiveness is 1, then the agent achieved all its intentions. If effectiveness is 0, then the agent failed to achieve any of its intentions. The key results of Kinny and Georgeff were as follows.

BOLD AGENT

CAUTIOUS AGENT

- If the rate of world change is low (i.e. the environment does not change quickly), then bold agents do well compared with cautious ones. This is because cautious ones waste time reconsidering their commitments while bold agents are busy working towards – and achieving – their intentions.
- If the rate of world change is high (i.e. the environment changes frequently), then cautious agents tend to outperform bold agents. This is because they are able to recognize

when intentions are doomed, and are also able to take advantage of serendipitous situations and new opportunities when they arise.

The bottom line is that different environment types require different intention reconsideration and commitment strategies. In static environments, agents that are strongly committed to their intentions will perform well. But in dynamic environments, the ability to react to changes by modifying intentions becomes more important, and weakly committed agents will tend to outperform bold agents.

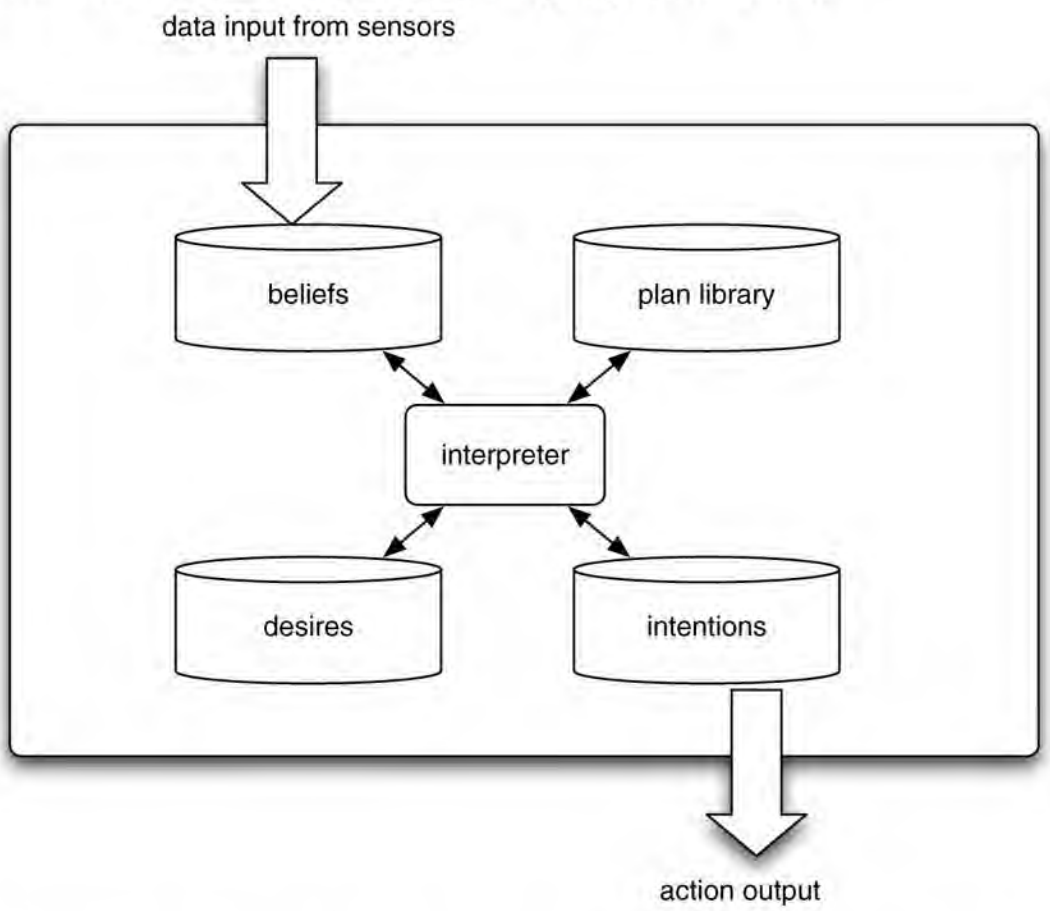
4.4 The Procedural Reasoning System

The Procedural Reasoning System (PRS), originally developed at Stanford Research Institute by Michael Georgeff and Amy Lansky, was perhaps the first agent architecture to explicitly embody the belief–desire–intention paradigm, and proved to be the most durable agent architecture developed to date. It has been applied in several of the most significant multiagent applications so far built, including an air-traffic control system called OASIS that is currently undergoing field trials at Sydney airport, a simulation system for the Royal Australian Air Force called SWARMM, and a business process management system called SPOC (Single Point of Contact), that is currently being marketed by Agentis Solutions [Georgeff and Rao, 1996].

PRS

An illustration of the PRS architecture is given in Figure 4.4. The PRS is often referred to as a BDI architecture, because it contains explicitly represented data structures loosely corresponding to these mental states [Wooldridge, 2000b].

Figure 4.4: The Procedural Reasoning System (PRS).



In the PRS, an agent does no planning from first principles. Instead, it is equipped with a

library of pre-compiled plans. These plans are manually constructed, in advance, by the agent programmer. Plans in the PRS each have the following components:

- a *goal* – the postcondition of the plan
- a *context* – the precondition of the plan
- a *body* – the 'recipe' part of the plan – the course of action to carry out.

The goal and context part of PRS plans are fairly conventional, but the body is slightly unusual. In the plans that we saw earlier in this chapter, the body of a plan was simply a sequence of actions. Executing the plan involved executing each action in turn. Such plans are possible in the PRS, but much richer kinds of plans are also possible. The first main difference is that, as well as having individual primitive actions as the basic components of plans, it is possible to have *goals*. The idea is that, when a plan includes a goal at a particular point, this means that this goal must then be achieved at this point before the remainder of the plan can be executed. It is also possible to have disjunctions of goals ('achieve φ or achieve ψ'), and loops ('keep achieving φ until ψ'), and so on.

At startup time a PRS agent will have a collection of such plans, and some initial beliefs about the world. Beliefs in the PRS are represented as Prolog-like facts – essentially, as atoms of first-order logic, in exactly the same way that we saw in deductive agents in the preceding chapter. In addition, at startup, the agent will typically have a top-level goal. This goal acts in a rather similar way to the 'main' method in Java or C.

When the agent starts up, the goal to be achieved is pushed onto a stack, called the *intention stack*. This stack contains all the goals that are pending achievement. The agent then searches through its plan library to see what plans have the goal on the top of the intention stack as their postcondition. Of these, only some will have their precondition satisfied, according to the agent's current beliefs. The set of plans that (i) achieve the goal, and (ii) have their precondition satisfied, become the possible *options* for the agent (cf. the *options* function described earlier in this chapter).

INTENTION STACK

The process of selecting between different possible plans is, of course, deliberation, a process that we have already discussed above. There are several ways of deliberating between competing options in PRS-like architectures. In the original PRS, deliberation is achieved by the use of *meta-level plans*. These are literally plans about plans. They are able to modify an agent's intention structures at run-time, in order to change the focus of the agent's practical reasoning. However, a simpler method is to use *utilities* for plans. These are numerical values; the agent simply chooses the plan that has the highest utility.

META-LEVEL PLANS

The chosen plan is then executed in its turn; this may involve pushing further goals onto the intention stack, which may then in turn involve finding more plans to achieve these goals, and so on. The process bottoms out with individual actions that may be directly computed (e.g. simple numerical calculations). If a particular plan to achieve a goal fails, then the agent is able to select another plan to achieve this goal from the set of all candidate plans.

To illustrate all this, Figure 4.5 shows a fragment of a Jam system [Huber, 1999]. Jam is a

second-generation descendant of the PRS, implemented in Java. The basic ideas are identical. The top-level goal for this system, which is another Blocks World example, is to have achieved the goal `blocks_stacked`. The initial beliefs of the agent are spelled out in the `FACTS` section. Expressed in conventional logic notation, the first of these is *On*(*Block*5, *Block*4), i.e.'block 5 is on top of block 4'.

The system starts by pushing the goal `blocks_stacked` onto the intention stack. The agent must then find a candidate plan for this; there is just one plan that has this as a `GOAL:` the 'top-level plan'. The context of this plan is empty, that is to say, true, and so this plan can be directly executed. Executing the body of the plan involves pushing the following goal onto the intention stack:

On(*block*3, *table*).

This is immediately achieved, as it is a FACT. The second subgoal is then posted:

On(*block*2, *block*3).

To achieve this, the 'stack blocks that are clear' plan is used; the first subgoals involve clearing both *block*2 and *block*3, which in turn will be done by two invocations of the 'clear a block' plan. When this is done, the move action is directly invoked to move *block*2 onto *block*3.

I leave the detailed behaviour as an exercise.

Figure 4.5: The Blocks World in Jam.

```
GOALS:
  ACHIEVE blocks_stacked;

FACTS:
  FACT ON "Block5" "Block4";      FACT ON "Block4" "Block3";
  FACT ON "Block1" "Block2";      FACT ON "Block2" "Table";
  FACT ON "Block3" "Table";       FACT CLEAR "Block1";
  FACT CLEAR "Block5";            FACT CLEAR "Table";

Plan: {
  NAME: "Top-level plan"
  GOAL: ACHIEVE blocks_stacked;
  CONTEXT:
  BODY:     ACHIEVE ON "Block3" "Table";
            ACHIEVE ON "Block2" "Block3";
            ACHIEVE ON "Block1" "Block2";
}

Plan: {
  NAME: "Stack blocks that are already clear"
  GOAL: ACHIEVE ON $OBJ1 $OBJ2;
  CONTEXT:
  BODY:     ACHIEVE CLEAR $OBJ1;
            ACHIEVE CLEAR $OBJ2;
            PERFORM move $OBJ1 $OBJ2;
  UTILITY: 10;
  FAILURE: EXECUTE print "\n\nStack blocks failed!\n\n";
}

Plan: {
  NAME: "Clear a block"
```

```
    GOAL: ACHIEVE CLEAR $OBJ;
    CONTEXT: FACT ON $OBJ2 $OBJ;
    BODY:     ACHIEVE ON $OBJ2 "Table";
    EFFECTS: RETRACT ON $OBJ1 $OBJ;
    FAILURE: EXECUTE print "\n\nClearing block failed!\n\n";
}
```

Notes and Further Reading

Some reflections on the origins of the BDI model, and on its relationship to other models of agency, may be found in [Georgeff et al., 1999]. Belief–desire–intention architectures originated in the work of the Rational Agency project at Stanford Research Institute in the mid 1980s. Key figures were Michael Bratman, Phil Cohen, Michael Georgeff, David Israel, Kurt Konolige, and Martha Pollack. The origins of the model lie in the theory of human practical reasoning developed by the philosopher Michael Bratman [Bratman, 1987], which focuses particularly on the role of intentions in practical reasoning. The conceptual framework of the BDI model is described in [Bratman et al., 1988], which also describes a specific BDI agent architecture called IRMA.

The best-known implementation of the BDI model is the PRS system, developed by Georgeff and colleagues [Georgeff and Ingrand, 1989; Georgeff and Lansky, 1987]. The PRS has been reimplemented several times since the mid 1980s, for example in the Australian AI Institute's DMARS system [d'Inverno et al., 1997], the University of Michigan's C++ implementation UM-PRS, and a Java version called Jam! [Huber, 1999]. Jack is a commercially available programming language, which extends the Java language with a number of BDI features [Busetta et al., 2000].

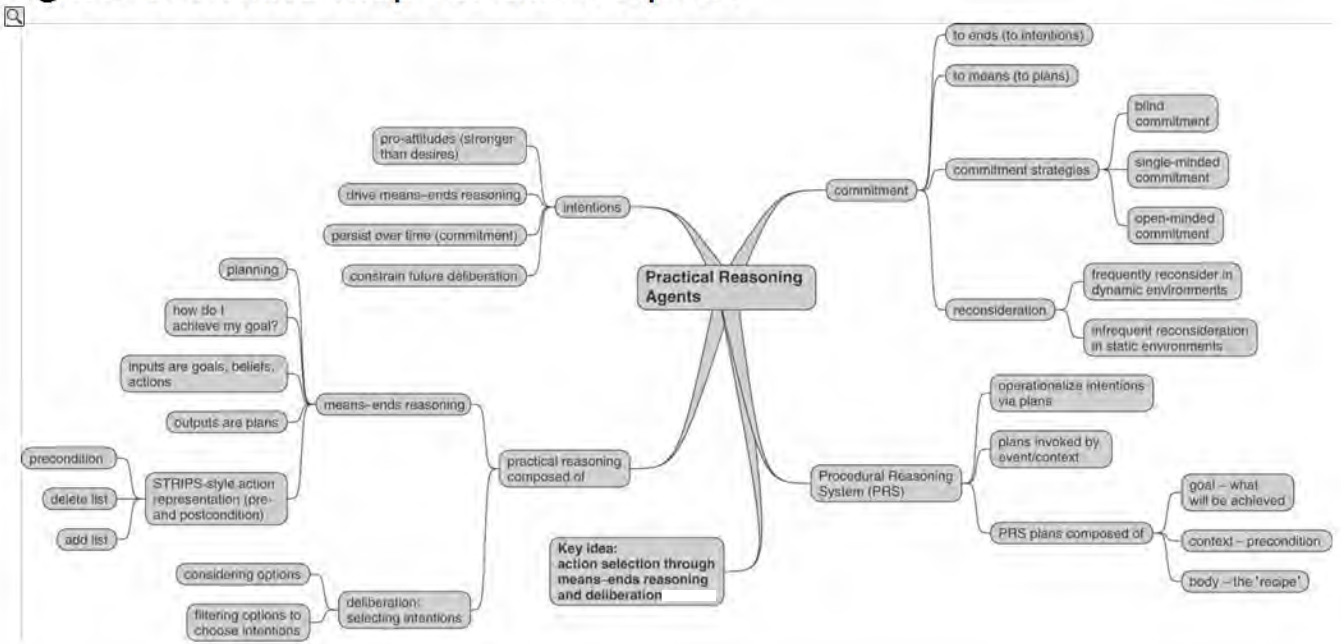
The description of the BDI model given here draws upon [Bratman et al., 1988] and [Rao and Georgeff, 1992], but is not strictly faithful to either. The most obvious difference is that I do not incorporate the notion of the 'filter override' mechanism described in [Bratman et al., 1988], and I also assume that plans are linear sequences of actions (which is a fairly 'traditional' view of plans), rather than the hierarchically structured collections of goals used by PRS. The BDI model is also interesting because a great deal of effort has been devoted to formalizing it. In particular, Anand Rao and Michael Georgeff have developed a range of BDI logics, which they use to axiomatize properties of BDI-based practical reasoning agents [Rao, 1996b; Rao and Georgeff, 1991a,b, 1992, 1993; Rao et al., 1992]. These models have been extended by others to deal with, for example, communication between agents [Haddadi, 1996].

An excellent discussion on the role of plans in practical reasoning is Martha Pollack's 1991 *Computers and Thought* award lecture, presented at the IJCAI-91 conference in Sydney, Australia, and published as 'The Uses of Plans' [Pollack, 1992]. Another article, which focuses on the distinction between 'plans as recipes' and 'plans as mental states' is [Pollack, 1990].

The best book on planning that I am aware of at the time of writing is [Ghallab et al., 2004], which gives an excellent overview of the history and state of the art of planning systems. The older [Allen et al., 1990] collects together most of the key papers from the planning literature up to 1990.

Class reading: [Bratman et al., 1988]. This is an interesting, insightful article, with not too much technical content. It introduces the IRMA architecture for practical reasoning agents, which has been very influential in the design of subsequent systems.

Figure 4.6: Mind map for this chapter.



Chapter 5 Reactive and Hybrid Agents

The many problems with symbolic/logical approaches to building agents led some researchers to question, and ultimately reject, the assumptions upon which such approaches are based. These researchers have argued that minor changes to the symbolic approach, such as weakening the logical representation language, will not be sufficient to build agents that can operate in time-constrained environments: nothing less than a whole new approach is required. In the mid to late 1980s, these researchers began to investigate alternatives to the symbolic AI paradigm. It is difficult to neatly characterize these different approaches, since their advocates are united mainly by a rejection of symbolic AI, rather than by a common manifesto. However, certain themes do recur:

- the rejection of symbolic representations, and of decision-making based on syntactic manipulation of such representations (such as logical reasoning)
- the idea that intelligent, rational behaviour is seen as innately linked to the *environment* an agent occupies – intelligent behaviour is not disembodied, but is a product of the *interaction* the agent maintains with its environment
- the idea that intelligent behaviour *emerges* from the interaction of various simpler behaviours.

EMERGENT BEHAVIOUR

5.1 Reactive Agents

Alternative approaches to agency are sometimes referred to as *behavioural* (since a common theme is that of developing and combining individual behaviours), *situated* (since a common theme is that of agents actually situated in some environment, rather than being disembodied from it), and finally – the term used in this chapter – *reactive* (because such systems are often perceived as simply reacting to an environment, without reasoning about it).

BEHAVIOURAL AGENT

SITUATED AGENT

5.1.1 The subsumption architecture

This section presents a survey of the *subsumption architecture*, which is arguably the best-known reactive agent architecture. It was developed by Rodney Brooks – one of the most vocal and influential critics of the symbolic approach to agency to have emerged in recent years. Brooks has propounded three key theses that have guided his work as follows [Brooks, 1991a,b].

SUBSUMPTION ARCHITECTURE

1. Intelligent behaviour can be generated *without* explicit representations of the kind that symbolic AI proposes.
2. Intelligent behaviour can be generated *without* explicit abstract reasoning of the kind that symbolic AI proposes.

3. Intelligence is an *emergent* property of certain complex systems.

Brooks also identifies two key ideas that have informed his research.

> **Situatedness and embodiment** 'Real' intelligence is situated in the world, not in disembodied systems such as theorem provers or expert systems.
>
> **Intelligence and emergence** 'Intelligent' behaviour arises as a result of an agent's interaction with its environment. Also, intelligence is 'in the eye of the beholder' – it is not an innate, isolated property.

These ideas were made concrete in the subsumption architecture. There are two defining characteristics of the subsumption architecture. The first is that an agent's decision-making is realized through a set of *task-accomplishing behaviours;* each behaviour may be thought of as an individual action selection function, which continually takes perceptual input and maps it to an action to perform. Each of these behaviour modules is intended to achieve some particular task. In Brooks's implementation, the behaviour modules are finite-state machines. An important point to note is that these task-accomplishing modules are assumed to include *no* complex symbolic representations, and are assumed to do *no* symbolic reasoning at all. In many implementations, these behaviours are implemented as rules of the form

TASK-ACCOMPLISHING BEHAVIOUR

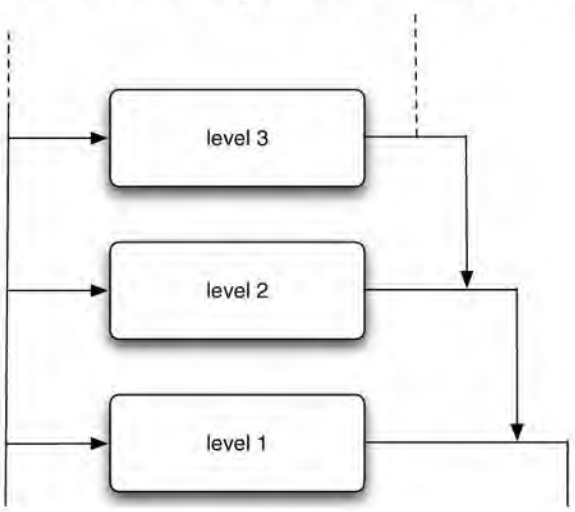
$$\text{situation} \rightarrow \text{action},$$

which simply map perceptual input directly to actions.

The second defining characteristic of the subsumption architecture is that many behaviours can 'fire' simultaneously. There must obviously be a mechanism to choose between the different actions selected by these multiple actions. Brooks proposed arranging the modules into a *subsumption hierarchy*, with the behaviours arranged into *layers*. Lower layers in the hierarchy are able to *inhibit* higher layers; the lower a layer is, the higher is its priority. The idea is that higher layers represent more abstract behaviours. For example, one might desire a behaviour 'avoid obstacles' in a mobile robot. It makes sense to give obstacle avoidance a high priority – hence this behaviour will typically be encoded in a *low-level* layer, which has *high* priority. The general organization of layers is illustrated in Figure 5.1.

SUBSUMPTION HIERARCHY

Figure 5.1: Action selection in layered architectures.



In implemented subsumption architecture systems, there is assumed to be quite tight coupling between perception and action – raw sensor input is not processed or transformed much, and there is certainly no attempt to transform images to symbolic representations. Action selection is realized through a set of behaviours, together with an *inhibition* relation holding between these behaviours. We write $b_1 \prec b_2$, and read this as 'b_1 inhibits b_2', that is, b_1 is lower in the hierarchy than b_2, and will hence get priority over b_2.

Overall, action selection proceeds by first computing the set of all behaviours that fire. Then, each behaviour that fires is checked, to determine whether there is some other higherpriority behaviour that fires. If not, then the behaviour is selected. If no behaviour fires, then the distinguished action *null* will be returned, indicating that no action has been chosen.

Given that one of our main concerns with logic-based decision-making was its theoretical complexity, it is worth pausing to examine how well our simple behaviour-based system performs. In practice, we can encode the decision-making logic of the subsumption architecture into hardware, giving *constant* decision time. For modern hardware, this means that an agent can be guaranteed to select an action within microseconds. Perhaps more than anything else, this computational simplicity is the strength of the subsumption architecture.

Steels's Mars explorer experiments

To illustrate the subsumption architecture in more detail, we will look at the following example (adapted from [Steels, 1990]).

> The objective is to explore a distant planet – more concretely, to collect samples of a particular type of precious rock. The location of the rock samples is not known in advance, but they are typically clustered in certain spots. A number of autonomous vehicles are available that can drive around the planet collecting samples and later re-enter a mother ship spacecraft to go back to Earth. There is no detailed map of the planet available, although it is known that the terrain is full of obstacles – hills, valleys, etc. – which prevent the vehicles from exchanging any communication.

The problem we are faced with is that of building an agent control architecture for each vehicle, so that they will cooperate to collect rock samples from the planet surface as efficiently as possible. Luc Steels argues that logic-based agents, of the type we described above, are 'entirely unrealistic' for this problem [Steels, 1990]. Instead, he proposes a solution using the subsumption architecture.

The solution makes use of two mechanisms introduced by Steels. The first is a *gradient field*. In order that agents can know in which direction the mother ship lies, the mother ship generates a radio signal. Now this signal will obviously weaken as distance from the source increases – to find the direction of the mother ship, an agent need therefore only travel 'up the gradient' of signal strength. The signal need not carry any information – it need only exist.

> GRADIENT FIELD

The second mechanism enables agents to communicate with one another. The characteristics of the terrain prevent direct communication (such as message passing), so Steels adopted an *indirect* communication method. The idea is that agents will carry 'radioactive crumbs', which can be dropped, picked up, and detected by passing robots. Thus if an agent drops some of these crumbs in a particular location, then later another agent happening upon this location will be able to detect them. This simple mechanism enables a quite sophisticated form of cooperation.

The behaviour of an individual agent is then built up from a number of behaviours, as we indicated above. First, we will see how agents can be programmed to *individually* collect samples. We will then see how agents can be programmed to generate a *cooperative* solution.

For individual (non-cooperative) agents, the lowest-level behaviour (and hence the behaviour with the highest 'priority') is obstacle avoidance. This behaviour can be represented in the rule:

if detect an obstacle then change direction. (5.1)

The second behaviour ensures that any samples carried by agents are dropped back at the mother ship:

if carrying a sample and at the base then drop sample; (5.2)

if carrying a sample and not at the base then travel up gradient. (5.3)

Behaviour (5.3) ensures that agents carrying samples will return to the mother ship (by heading towards the origin of the gradient field). The next behaviour ensures that agents will collect samples they find:

if detect a sample then pick sample up. (5.4)

The final behaviour ensures that an agent with 'nothing better to do' will explore randomly:

if true then move randomly. (5.5)

The precondition of this rule is thus assumed to always fire. These behaviours are arranged into the following hierarchy:

$$(5.1) \prec (5.2) \prec (5.3) \prec (5.4) \prec (5.5).$$

The subsumption hierarchy for this example ensures that, for example, an agent will *always* turn if any obstacles are detected; if the agent is at the mother ship and is carrying samples, then it will *always* drop them if it is not in any immediate danger of crashing, and so on. The 'top level' behaviour – a random walk – will only ever be carried out if the agent has nothing more urgent to do. It is not difficult to see how this simple set of behaviours will solve the problem: agents will search for samples (ultimately by searching randomly), and when they find them, will return them to the mother ship.

If the samples are distributed across the terrain entirely at random, then equipping a large number of robots with these very simple behaviours will work extremely well. But we know from the problem specification, above, that this is not the case: the samples tend to be

located in clusters. In this case, it makes sense to have agents *cooperate* with one another in order to find the samples. Thus when one agent finds a large sample, it would be helpful for it to communicate this to the other agents, so that they can help it collect the rocks. Unfortunately, we also know from the problem specification that *direct* communication is impossible. Steels developed a simple solution to this problem, partly inspired by the foraging behaviour of ants. The idea revolves around an agent creating a 'trail' of radioactive crumbs whenever it finds a rock sample. The trail will be created when the agent returns the rock samples to the mother ship. If at some later point another agent comes across this trail, then it need only follow it down the gradient field to locate the source of the rock samples. Some small refinements improve the efficiency of this ingenious scheme still further. First, as an agent follows a trail to the rock sample source, it picks up some of the crumbs it finds, hence making the trail fainter. Secondly, the trail is *only* laid by agents returning to the mother ship. Hence if an agent follows the trail out to the source of the nominal rock sample only to find that it contains no samples, it will reduce the trail on the way out, and will not return with samples to reinforce it. After a few agents have followed the trail to find no sample at the end of it, the trail will in fact have been removed.

The modified behaviours for this example are as follows. Obstacle avoidance (5.1) remains unchanged. However, the two rules determining what to do if carrying a sample are modified as follows:

if carrying a sample and at the base then drop sample; (5.6)

if carrying a sample and *not* at the base (5.7)

then drop 2 crumbs *and* travel up gradient.

The behaviour (5.7) requires an agent to drop crumbs when returning to base with a sample, thus either reinforcing or creating a trail. The 'pick up sample' behaviour (5.4) remains unchanged. However, an additional behaviour is required for dealing with crumbs:

if sense crumbs then pick up 1 crumb and travel down gradient. (5.8)

Finally, the random movement behaviour (5.5) remains unchanged. These behaviours are then arranged into the following subsumption hierarchy:

$$(5.1) \prec (5.6) \prec (5.7) \prec (5.4) \prec (5.8) \prec (5.5).$$

Steels shows how this simple adjustment achieves near-optimal performance in many situations. Moreover, the solution is *cheap* (the computing power required by each agent is minimal) and *robust* (the loss of a single agent will not affect the overall system significantly).

5.1.2 PENGI

Agre observed that most everyday activity is 'routine' in the sense that it requires little – if any – new abstract reasoning. Most tasks, once learned, can be accomplished in a routine way, with little variation. Agre proposed that an efficient agent architecture could be based on the idea of 'running arguments'. Crudely, the idea is that, as most decisions are routine, they can be encoded into a low-level structure (such as a digital circuit), which only needs periodic updating, perhaps to handle new kinds of problems. His approach was illustrated with the

updating, perhaps to handle new kinds of problems. His approach was illustrated with the celebrated PENGI system [Agre and Chapman, 1987]. PENGI is a simulated computer game, with the central character controlled using a scheme such as that outlined above.

5.1.3 Situated automata

Rosenschein and Kaelbling pointed out that, just because we might conceptualize, or specify the behaviour of an agent in terms of concepts like beliefs and goals, and indeed might use a logical representation of beliefs and goals to specify the agent, this does not imply that the agent must be *implemented* as a theorem prover for a logic of beliefs and goals. In their *situated automata* paradigm, an agent is specified in a logic of beliefs and goals, but this specification is then compiled down to a digital machine, which satisfies the beliefs – goal specification in a precise formal sense [Kaelbling, 1991; Kaelbling and Rosenschein, 1990; Rosenschein, 1985; Rosenschein and Kaelbling, 1986]. The resulting digital machine can operate in a provably time-bounded fashion; it does not do any symbol manipulation, and in fact no symbolic expressions are represented in the machine at all. The logic used to specify an agent is essentially a logic of knowledge:

SITUATED AUTOMATA

> [An agent] … x is said to carry the information that p in world state s, written $s \models K(x, p)$, if for all world states in which x has the same value as it does in s, the proposition p is true.
>
> [Kaelbling and Rosenschein, 1990, p. 36]

An agent is specified in terms of two components: perception and action. Two programs are then used to synthesize agents: `RULER` is used to specify the perception component of an agent; `GAPPS` is used to specify the action component.

RULER takes as its input three components:

> [A] specification of the semantics of the [agent's] inputs ("whenever bit 1 is on, it is raining"); a set of static facts ("whenever it is raining, the ground is wet"); and a specification of the state transitions of the world ("if the ground is wet, it stays wet until the sun comes out"). The programmer then specifies the desired semantics for the output ("if this bit is on, the ground is wet"), and the compiler … [synthesizes] a circuit whose output will have the correct semantics. … All that declarative "knowledge" has been reduced to a very simple circuit.
>
> [Kaelbling, 1991, p. 86]

The `GAPPS` program takes as its input a set of *goal reduction rules* (essentially rules that encode information about how goals can be achieved), and a top-level goal, and generates a program that can be translated into a digital circuit in order to realize the goal. Once again, the generated circuit does not represent or manipulate symbolic expressions; all symbolic manipulation is done at compile time.

The situated automata paradigm has attracted much interest, as it appears to combine the best elements of both reactive and symbolic declarative systems. However, at the time of writing, the theoretical limitations of the approach are not well understood; there are similarities with the automatic synthesis of programs from temporal logic specifications, a complex area of much ongoing work in mainstream computer science (see the comments in [Emerson, 1990]).

much ongoing work in mainstream computer science (see the comments in [Emerson, 1990]).

5.1.4 The agent network architecture

Pattie Maes has developed an agent architecture in which an agent is defined as a set of *competence modules* [Maes, 1989, 1990b, 1991]. These modules loosely resemble the behaviours of Brooks's subsumption architecture (above). Each module is specified by the designer in terms of pre- and post-conditions (rather like `STRIPS` operators), and an *activation level*, which gives a real-valued indication of the *relevance* of the module in a particular situation. The higher the activation level of a module, the more likely it is that this module will influence the behaviour of the agent. Once specified, a set of competence modules is compiled into a *spreading activation network*, in which the modules are linked to one another in ways defined by their pre- and post-conditions. For example, if module a has postcondition ϕ, and module b has precondition ϕ, then a and b are connected by a *successor* link. Other types of link include predecessor links and conflicter links. When an agent is executing, various modules may become more active in given situations, and may be executed. The result of execution may be a command to an effector unit, or perhaps the increase in activation level of a successor module.

There are obvious similarities between the agent network architecture and neural network architectures. Perhaps the key difference is that it is difficult to say what the meaning of a node in a neural net is; it only has a meaning in the context of the net itself. Since competence modules are defined in declarative terms, however, it is very much easier to say what their meaning is.

5.1.5 The limitations of reactive agents

There are obvious advantages to reactive approaches such as Brooks's subsumption architecture: simplicity, economy, computational tractability, robustness against failure, and elegance all make such architectures appealing. But there are some fundamental, unsolved problems, not just with the subsumption architecture, but with other purely reactive architectures.

- If agents do not employ models of their environment, then they must have sufficient information available in their *local* environment to determine an acceptable action.
- Since purely reactive agents make decisions based on *local* information (i.e. information about the agents' *current* state), it is difficult to see how such decision-making could take into account *non-local* information – it must inherently take a 'short-term' view.
- One major selling point of purely reactive systems is that overall behaviour *emerges* from the interaction of the component behaviours when the agent is placed in its environment. But the very term 'emerges' suggests that the relationship between individual behaviours, environment, and overall behaviour is not understandable. This necessarily makes it very hard to *engineer* agents to fulfil specific tasks. Ultimately, there is no principled *methodology* for building such agents; one must use a laborious process of experimentation, trial, and error to engineer an agent.
- While effective agents can be generated with small numbers of behaviours (typically fewer than ten layers), it is *much* harder to build agents that contain many layers. The dynamics of the interactions between the different behaviours become too complex to understand.

Various solutions to these problems have been proposed. One of the most popular of these is the idea of *evolving* agents to perform certain tasks. This area of work has largely broken away from the mainstream AI tradition in which work on, for example, logic-based agents is carried out, and is documented primarily in the *artificial life* (alife) literature.

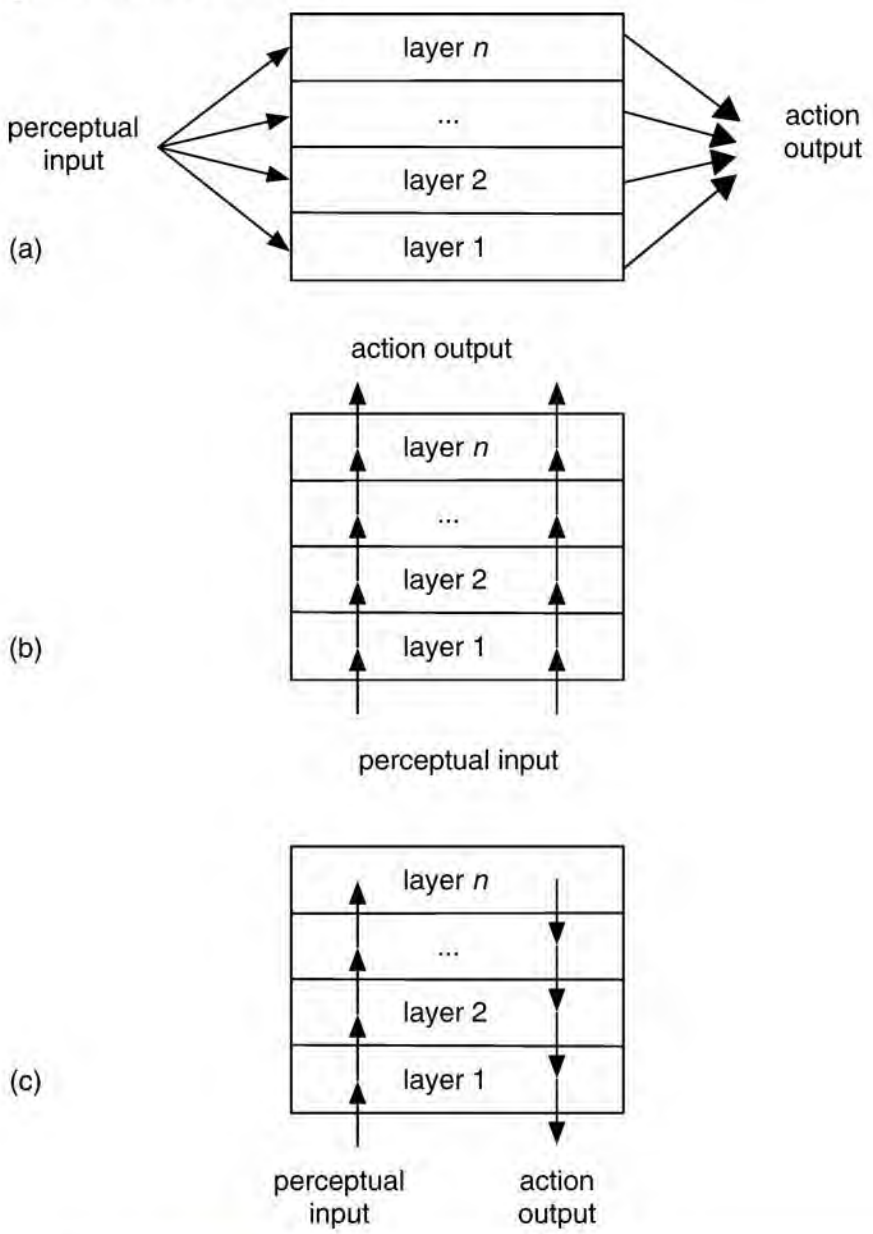
5.2 Hybrid Agents

Given the requirement that an agent be capable of reactive and proactive behaviour, an obvious decomposition involves creating separate subsystems to deal with these different types of behaviours. This idea leads naturally to a class of architectures in which the various subsystems are arranged into a hierarchy of interacting *layers*. In this section, we will consider some general aspects of layered architectures, and then go on to consider two examples of such architectures: InteRRaP and TouringMachines.

LAYERED ARCHITECTURE

Typically, there will be at least two layers: to deal with reactive and proactive behaviours, respectively. In principle, there is no reason why there should not be many more layers.

Figure 5.2: Information and control flows in three types of layered agent architecture.



(Source: [Müller et al., 1995, p. 263].)

(Source: [Müller et al., 1995, p. 265].)

It is useful to characterize such architectures in terms of the information and control flows within the layers. Broadly speaking, we can identify two types of control flow within layered architectures as follows (see Figure 5.2).

Horizontal layering In horizontally layered architectures (Figure 5.2(a)), the software layers are each directly connected to the sensory input and action output. In effect, each layer itself acts like an agent, producing suggestions as to what action to perform.

Vertical layering In vertically layered architectures (see parts (b) and (c) of Figure 5.2), sensory input and action output are each dealt with by at most one layer.

The great advantage of horizontally layered architectures is their conceptual simplicity: if we need an agent to exhibit n different types of behaviour, then we implement n different layers. However, because the layers are each in effect competing with one another to generate action suggestions, there is a danger that the *overall* behaviour of the agent will not be coherent. In order to ensure that horizontally layered architectures *are* consistent, they generally include a *mediator* function, which makes decisions about which layer has 'control' of the agent at any given time. The need for such central control is problematic; it means that the designer must potentially consider all possible interactions between layers. If there are n layers in the architecture, and each layer is capable of suggesting m possible actions, then this means there are m^n such interactions to be considered. This is clearly difficult from a design point of view in any but the most simple system. The introduction of a central control system also introduces a *bottleneck* into the agent's decision-making.

HORIZONTAL LAYERING

MEDIATOR FUNCTION

VERTICAL LAYERING

These problems are partly alleviated in a vertically layered architecture. We can subdivide vertically layered architectures into *one-pass* architectures (Figure 5.2(b)) and *two pass* architectures (Figure 5.2(c)). In one-pass architectures, control flows sequentially through each layer, until the final layer generates action output. In two-pass architectures, information flows up the architecture (the first pass) and control then flows back down. There are some interesting similarities between the idea of two-pass vertically layered architectures and the way that organizations work, with information flowing up to the highest levels of the organization, and commands then flowing down. In both one-pass and two-pass vertically layered architectures, the complexity of interactions between layers is reduced: since there are $n - 1$ interfaces between n layers, then if each layer is capable of suggesting m actions, there are at most $m^2(n - 1)$ interactions to be considered between layers. This is clearly much simpler than the horizontally layered case. However, this simplicity comes at the cost of some flexibility: in order for a vertically layered architecture to make a decision, control must pass between *each* different layer. This is not fault tolerant: failures in any one layer are likely to have serious consequences for agent performance.

ONE-PASS ARCHITECTURE

TWO-PASS ARCHITECTURE

In the remainder of this section, we will consider two examples of layered architectures: Innes Ferguson's TouringMachines, and Jörg Müller's InteRRaP. The former is an example of a horizontally layered architecture; the latter is a (two-pass) vertically layered architecture.

5.2.1 TouringMachines

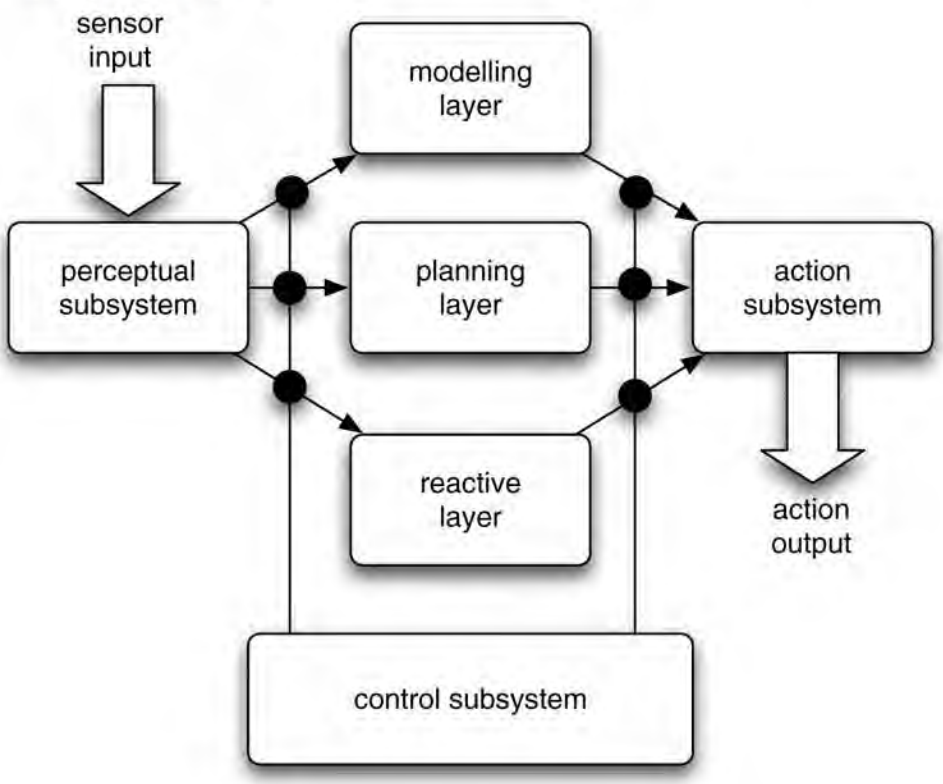
TOURINGMACHINES ARCHITECTURE

The TouringMachines architecture is illustrated in Figure 5.3. As this figure shows, Touring-Machines consist of three *activity-producing layers*. That is, each layer continually produces suggestions for what actions the agent should perform. The *reactive layer* provides a moreor-less immediate response to changes that occur in the environment. It is implemented as a set of situation–action rules, like the behaviours in Brooks's subsumption architecture (see Section 5.1.1). These rules map sensor input directly to effector output. The original demonstration scenario for TouringMachines was that of autonomous vehicles driving between locations through streets populated by other similar agents. In this scenario, reactive rules typically deal with functions like obstacle avoidance. For example, here is an example of a reactive rule for avoiding the kerb (from [Ferguson, 1992a, p. 59]):

ACTIVITY-PRODUCING LAYERS

REACTIVE LAYER

Figure 5.3: TouringMachines: a horizontally layered agent architecture.




```
rule-1: kerb-avoidance
```

```
if
  is-in-front (Kerb, Observer) and
   speed(Observer) > 0 and
   separation(Kerb, Observer) < KerbThreshHold
then
   change-orientation(KerbAvoidanceAngle)
```

Here change-orientation(...) is the action suggested if the rule fires. The rules can only make references to the agent's current state – they cannot do any explicit reasoning about the world, and on the right-hand side of rules are *actions*, not predicates. Thus if this rule fired, it would not result in any central environment model being updated, but would just result in an action being suggested by the reactive layer.

The TouringMachines *planning layer* achieves the agent's proactive behaviour. Specifically, the planning layer is responsible for the 'day-to-day' running of the agent – under normal circumstances, the planning layer will be responsible for deciding what the agent does. However, the planning layer does not do 'first-principles' planning. That is, it does not attempt to generate plans from scratch. Rather, the planning layer employs a *library* of plan 'skeletons' called *schemas*. These skeletons are in essence hierarchically structured plans, which the TouringMachines planning layer elaborates at run-time in order to decide what to do (cf. the PRS architecture discussed in Chapter 4). So, in order to achieve a goal, the planning layer attempts to find a schema in its library which matches that goal.

PLANNING LAYER

This schema will contain subgoals, which the planning layer elaborates by attempting to find other schemas in its plan library that match these subgoals.

MODELLING LAYER

The *modelling* layer represents the various entities in the world (including the agent itself, as well as other agents). The modelling layer thus predicts conflicts between agents, and generates new goals to be achieved in order to resolve these conflicts. These new goals are then posted down to the planning layer, which makes use of its plan library in order to determine how to satisfy them.

CONTROL SUBSYSTEM

The three control layers are embedded within a *control subsystem*, which is effectively responsible for deciding which of the layers should have control over the agent. This control subsystem is implemented as a set of *control rules*. Control rules can either *suppress* sensor information between the control rules and the control layers, or else *censor* action outputs from the control layers. Here is an example censor rule [Ferguson, 1995, p. 207]:

```
censor-rule-1:
    if
       entity(obstacle-6) in perception-buffer
    then
       remove-sensory-record(layer-R, entity(obstacle-6))
```

remove-sensory-record(layer-R, entity(obstacle-6))

This rule prevents the reactive layer from ever knowing about whether `obstacle-6` has been perceived. The intuition is that although the reactive layer will in general be the most appropriate layer for dealing with obstacle avoidance, there are certain obstacles for which other layers are more appropriate. This rule ensures that the reactive layer never comes to know about these obstacles.

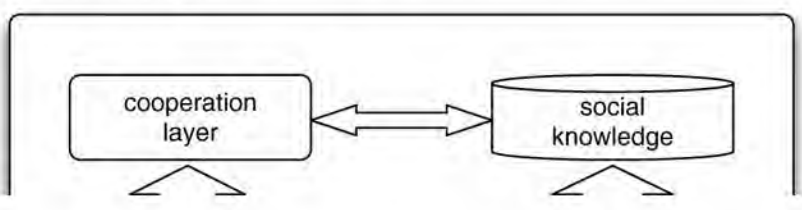
5.2.2 InteRRaP

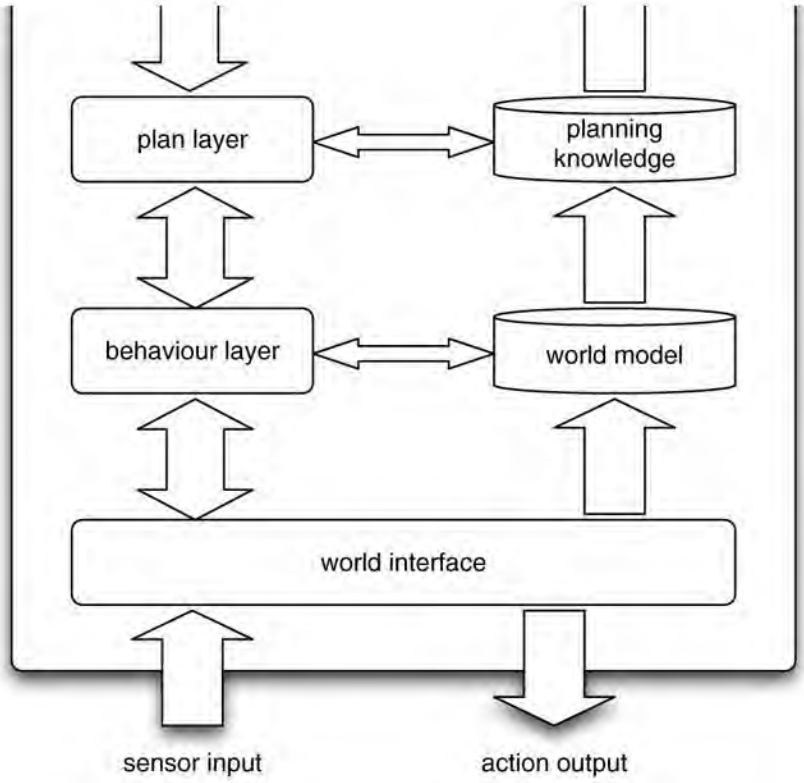
INTERRAP ARCHITECTURE

InteRRaP is an example of a vertically layered two-pass agent architecture – see Figure 5.4. As Figure 5.4 shows, InteRRaP contains three control layers, as in TouringMachines. Moreover, the purpose of each InteRRaP layer appears to be rather similar to the purpose of each corresponding TouringMachines layer. Thus the lowest (*behaviour-based*) layer deals with reactive behaviour; the middle (*local planning*) layer deals with everyday planning to achieve the agent's goals, and the uppermost (*cooperative planning*) layer deals with social interactions. Each layer has associated with it a *knowledge base*, i.e. a representation of the world appropriate for that layer. These different knowledge bases represent the agent and its environment at different levels of abstraction. Thus the highest-level knowledge base represents the plans and actions of other agents in the environment; the middle-level knowledge base represents the plans and actions of the agent itself; and the lowest-level knowledge base represents 'raw' information about the environment. The explicit introduction of these knowledge bases distinguishes TouringMachines from InteRRaP.

The way the different layers in InteRRaP conspire to produce behaviour is also quite different from TouringMachines. The main difference is in the way the layers interact with the environment. In TouringMachines, each layer was directly coupled to perceptual input and action output. This necessitated the introduction of a supervisory control framework, to deal with conflicts or problems between layers. In InteRRaP, layers interact with *each other* to achieve the same end. The two main types of interaction between layers are *bottom-up activation* and *top-down execution*. Bottom-up activation occurs when a lower layer passes control to a higher layer because it is not *competent* to deal with the current situation. Top-down execution occurs when a higher layer makes use of the facilities provided by a lower layer to achieve one of its goals. The basic flow of control in InteRRaP begins when perceptual input arrives at the lowest layer in the architecture. If the reactive layer can deal with this input, then it will do so; otherwise, bottom-up activation will occur, and control will be passed to the local planning layer. If the local planning layer can handle the situation, then it will do so, typically by making use of top-down execution. Otherwise, it will use bottom-up activation to pass control to the highest layer. In this way, control in InteRRaP will flow from the lowest layer to higher layers of the architecture, and then back down again.

Figure 5.4: InteRRaP – a vertically layered two-pass agent architecture.



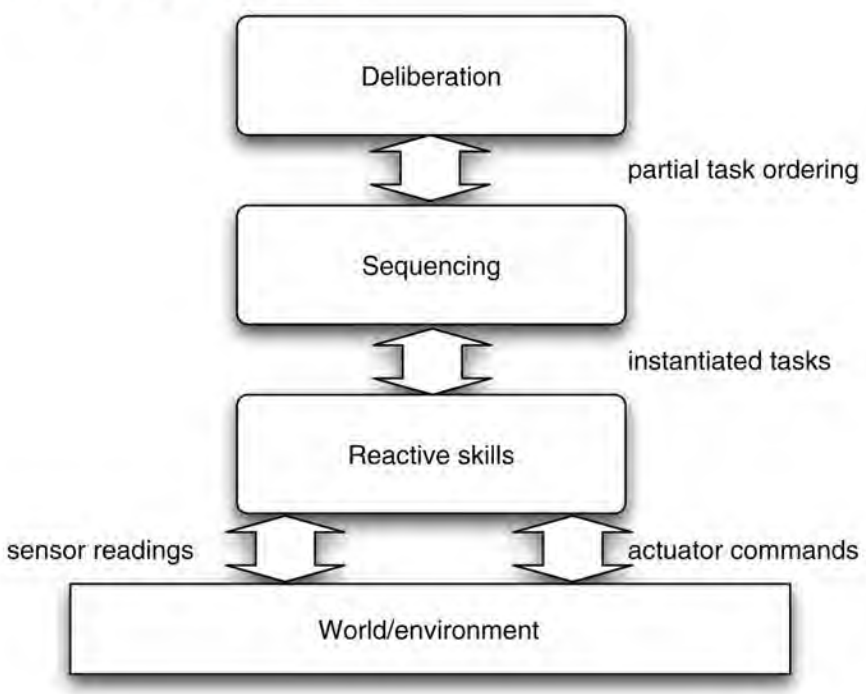


The internals of each layer are not important for the purposes of this chapter. However, it is worth noting that each layer implements two general functions. The first of these is a *situation recognition and goal activation* function, which maps a knowledge base (one of the three layers) and current goals to a new set of goals. The second function is responsible for *planning and scheduling* – it is responsible for selecting which plans to execute, based on the current plans, goals, and knowledge base of that layer.

SITUATION RECOGNITION

PLANNING AND SCHEDULING

Figure 5.5: The 3T architecture.



5.2.3 3T

Like TouringMachines and InteRRaP, the 3T architecture is another example of a three-level architecture for agent control. An overview of the architecture is given in Figure 5.5. The three layers in the 3T architecture are as follows (from least to most abstract):

3T ARCHITECTURE

- *reactive skills*

  REACTIVE SKILLS

- *skill sequencing*

  SKILL SEQUENCING

- *deliberation and high-level planning*.

  DELIBERATION AND PLANNING

A *skill* in the 3T architecture is a primitive behaviour that captures a particular kind of ability. For example, a skill might be the ability of a mobile robot to move from one location in a warehouse to another location. We can draw an analogy between skills and (for example) 'native methods' in Java. Ultimately, whatever a 3T agent does will involve the execution of particular skills. The skills layer thus contains a set of such skills, and, typically, one of these skills will be being executed at any given time. Skills are actually implemented in 3T using Firby's *reactive action packages* [Firby, 1987].

The *sequencing* layer of 3T is responsible for selecting and instantiating individual skills. Finally, the *planning* layer is responsible for synthesizing high-level plans for the agent, including deadlines for the completion of activity and plans for the allocation of resources.

The following example (from [Bonasso et al., 1997]) explains the role of the three layers in more detail:

> Imagine a repair robot charging in a docking bay on a space station. At the beginning of a typical day, there will be several routine maintenance tasks to perform on the outside of the station, such as retrieving broken items or inspecting power levels. In addition, a human supervisor assigns the robot a set of inspection and repair tasks at a number of sites around the station. The planner synthesizes all of these goals into a partially ordered plan listing tasks for the robot to perform. These tasks would call for the robot to move from site to site conducting the appropriate repair or inspection at each site. [For example] (1) navigate to the `camera-site-1`, (2) attach to `camera-site-1`, (3) unload a repaired camera, and (4) detach from `camera-site-1`. Each of these corresponds to one or more skills. … The [sequencing layer] activates a specific set of skills at the skill level. … The activated skills (reactive layer) will move the state of the world in a direction that should cause the desired [state of affairs].

5.2.4 Stanley

We conclude our survey of agent architectures by looking at the architecture of Stanley: the robot that won the DARPA Grand Challenge (see text box) [Thrun et al., 2007]. Stanley is an autonomous agent embodied in a car (a Volkswagen Touareg R5, essentially unmodified from

consumer versions of this vehicle apart from the changes necessary to allow computer control – see Figure 5.6). The software architecture of Stanley's control system contains about 30 different independently operating modules, roughly organized into six layers, as follows.

Sensor interface layer Stanley is equipped with a wide range of sensor apparatus: laser range finders, a camera, radar, and global positioning system, as well as sensors providing data about the state of the vehicle itself. The sensor interface layer is basically responsible for receiving and time-stamping this data. Not only does Stanley possess a number of different sensor types, but these sensors provide data at high frequency: for example, each of the five laser range finders provides data 100 times per second.

Perception layer The perception layer handles the problem of translating time-stamped data into the internal models used to represent Stanley's environment. The internal models that Stanley represents comprise information relating both to the state of the vehicle and the state of the environment. Stanley models the vehicle state in terms of 15 variables, relating to position, velocity, orientation, accelerometer, and gyro. The representation of the environment, however, is much more complex: terrain mapping and road identification are extremely complex processes.

The DARPA Grand Challenges

Most adults in the developed world think that driving a car is easy, or at least, they don't see the ability to drive a car as an indication of great intellect. But getting *computers* to drive cars turns out to be very difficult. Nevertheless, there are many obvious benefits to having computers drive cars, and so, to try to stimulate research in unmanned vehicles, DARPA (a US government research funding agency) decided to organize a *grand challenge*. The idea was to have a competition for unmanned vehicles to travel across more than 140 miles of rough terrain in California and Nevada. The fastest vehicle to be able to do this stood to win a $1 million prize for its designers. Note that, once they left the starting point, vehicles would be required to be *completely* autonomous, with all sensor data processing and decision-making on board. The first grand challenge took place on 13 March 2004. Fifteen teams entered, many from the world's great technical universities. The results were, to put it bluntly, a bit embarrassing. None of the entries made it more than 7.4 miles on the course; mechanical failure hit some, while others didn't even make it out of the starting area, and others had to be disabled by the organizers because they posed a risk. Undaunted, DARPA organized a follow-on grand challenge in 2005, with prize money doubled to $2 million. A total of 43 teams entered the qualifying rounds, and 23 finalists got to participate in the race itself, on 8 October 2005. This time, remarkably, no less than five teams successfully completed the 132-mile course in the Nevada desert: the winning robot, Stanley, designed by a Stanford University team under the supervision of Sebastian Thrun, completed the course in 6 hours 53 minutes, averaging a very respectable 19.2 miles per hour. It is hard to overstate the significance of these achievements. Many researchers now regard the problem of having computers drive cars as being essentially solved. Of course, the *social* problems associated with putting unmanned vehicles on the same roads as vehicles with human drivers are some way from being understood …

Planning and control layer The planning and control layer is responsible for path planning, steering, and throttle/brake control.

Vehicle interface layer This layer provides the interface between the control system and the actuators and vehicle controls.

User interface layer Provides a touch-screen control for starting up the system.

Global services layer Provides services used by all system modules (e.g. filestore, clock, inter-process communication).

It should be clear that in terms of complexity, Stanley is in a different league from the other agents we have discussed in this book, not least because many of those systems either inhabit software environments, or were only ever implemented in simulated environments. The key problem that Sebastian Thrun's team had to face when they built Stanley was not one of action: it was one of *perception*. If Stanley knows that it is about to hit an obstacle, then the decision to stop is easy. The difficult part is knowing that it is going to hit a wall. Thus most of the effort in building Stanley went into the sensors, and the associated techniques to interpret sensor data. These techniques are extremely complex, and considerably beyond the scope of this book – see [Thrun et al., 2005] for a detailed exposition.

Figure 5.6: Stanley: The autonomous robot that won the 2005 DARPA Grand Challenge, by driving itself safely across 142 miles of the Mojave desert. Reproduced with permission from the Stanford Racing Team.

Notes and Further Reading

Brooks's original paper on the subsumption architecture – the one that started all the fuss – was published as [Brooks, 1986]. The description and discussion here is partly based on [Ferber, 1996]. This original paper seems to be somewhat less radical than many of his later

ones [Brooks, 1990, 1991b]. The version of the subsumption architecture used in this chapter is actually a simplification of that presented by Brooks. The subsumption architecture is probably the best-known reactive architecture around – but there are many others. The collection of papers edited by [Maes, 1990a] contains papers that describe many of these, as does the collection by [Agre and Rosenschein, 1996]. Other approaches include:

- Nilsson's *teleo reactive programs* [Nilsson, 1992];
- Schoppers' *universal plans* – which are essentially decision trees that can be used to efficiently determine an appropriate action in any situation [Schoppers, 1987];
- Firby's *reactive action packages* [Firby, 1987].

[Kaelbling, 1986] gives a good discussion of the issues associated with developing resourcebounded rational agents, and proposes an agent architecture somewhat similar to that developed by Brooks.

[Ginsberg, 1989] gives a critique of reactive agent architectures based on cached plans; [Etzioni, 1993] gives a critique of the claim by Brooks that intelligent agents must be situated 'in the real world'. He points out that *software environments* (such as computer operating systems and computer networks) can provide a challenging environment in which agents can operate.

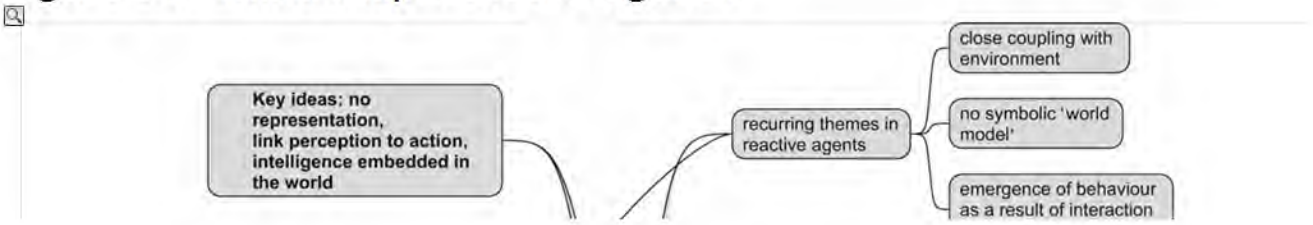
Layered architectures are currently the most popular general class of agent architecture available. Layering represents a natural decomposition of functionality: it is easy to see how reactive, proactive, and social behaviour can be generated by the reactive, proactive, and social layers in an architecture. The main problem with layered architectures is that, while they are arguably a *pragmatic* solution, they lack the conceptual and semantic clarity of unlayered approaches. In particular, while logic-based approaches have a clear logical semantics, it is difficult to see how such a semantics could be devised for a layered architecture. Another issue is that of interactions between layers. If each layer is an independent activity-producing process (as in TouringMachines), then it is necessary to consider all possible ways in which the layers can interact with one another. This problem is partly alleviated in a two-pass vertically layered architecture such as InteRRaP.

The introductory discussion of layered architectures given here draws upon [Müller et al., 1995, pp. 262–264]. The best references to TouringMachines are [Ferguson, 1992a,b]). The definitive reference to InteRRaP are [Müuller, 1997].

A survey of the entries in the 2005 DARPA Grand Challenge was published as [Buehler et al., 2007], while [Seetharaman et al., 2006] presents an overview.

Class reading: [**Brooks, 1986**]. A provocative, fascinating article, packed with ideas. It is interesting to compare this with some of Brooks's later – arguably more controversial – articles.

Figure 5.7: Mind map: reactive agents.



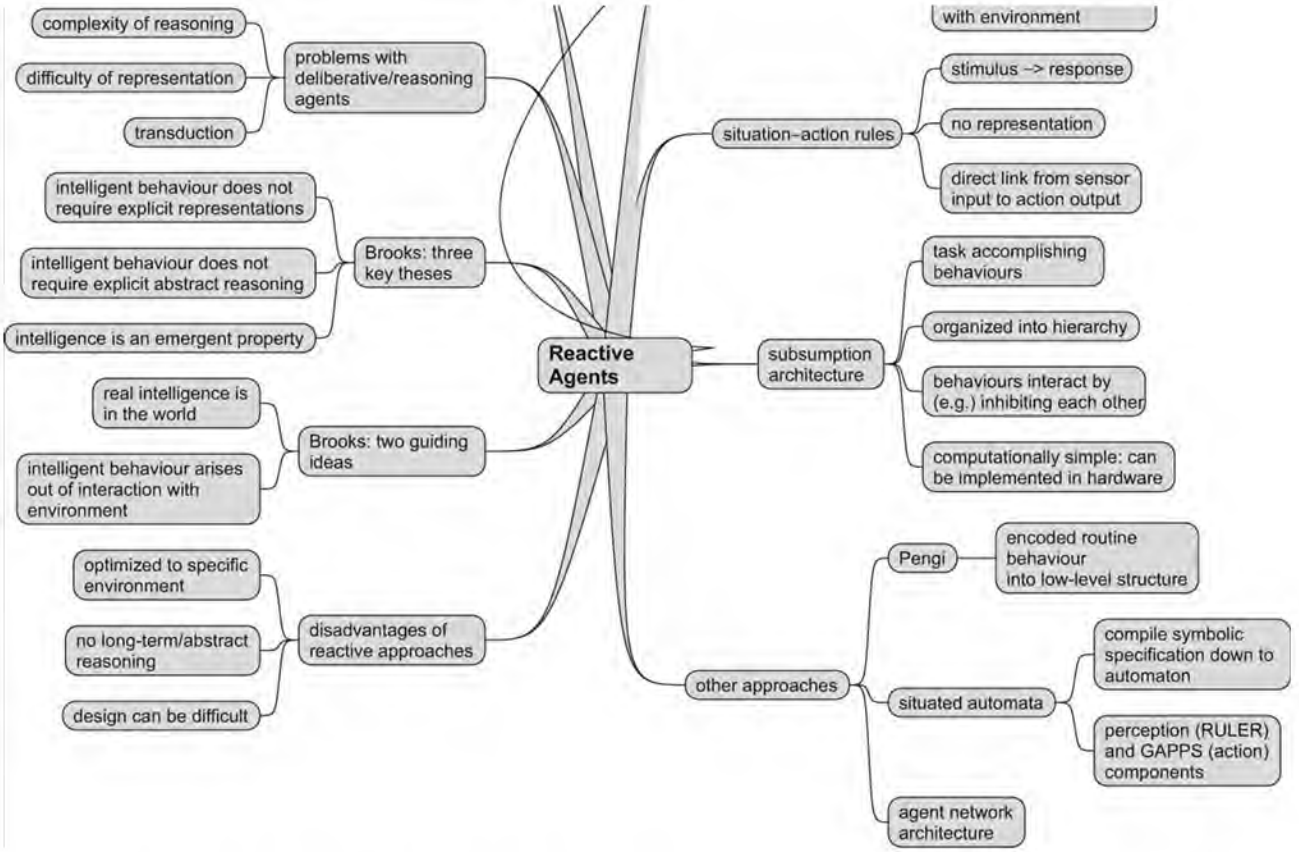
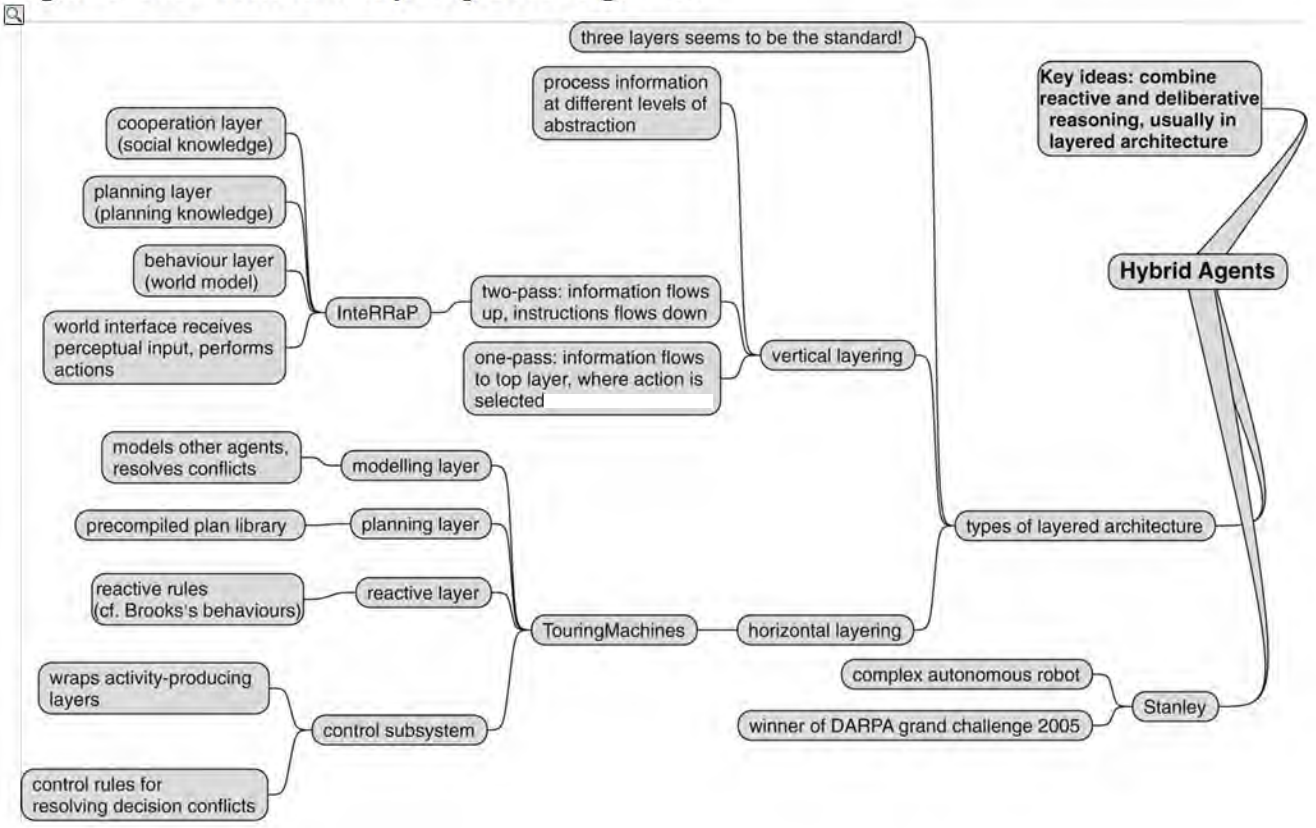


Figure 5.8: Mind map: hybrid agents.



Part III Communication and Cooperation

So, now you know how to build an agent. The next issue we consider is that of *how to put agents together*; we move from the *micro level* of individual agents, to the *macro level* of multiagent systems. Clearly, one of the main issues that we would expect to see is that of communication, and indeed communication does feature prominently in this part. However, there is an even more fundamental issue that must be addressed before agents can cooperate: that of ensuring that they can *understand each other*. So, we begin by looking at approaches to this problem, which is the area of *ontologies*. We then move on to consider communication languages – standardized languages that allow agents to exchange knowledge and request actions to be performed. Then we consider the issue of working together to solve problems. We conclude this part by looking at methodologies for building multiagent systems, and some applications of the approaches we have considered so far.

Chapter 6 Understanding Each Other

If two agents are to communicate about some domain, then it is necessary for them to agree on the *terminology* that they use to describe this domain. For example, imagine that an agent is buying a particular engineering item (nut or bolt) from another agent; the buyer needs to be able to unambiguously specify to the seller the desired properties of the item, such as its size. The agents thus need to be able to agree both on what 'size' means, and also on what terms like 'inch' or 'centimetre' mean. An *ontology* is a specification of a set of terms, intended to provide a common basis of understanding about some domain:

ONTOLOGY

> An ontology is a formal definition of a body of knowledge. The most typical type of ontology used in building agents involves a structural component. Essentially a taxonomy of class and subclass relations coupled with definitions of the relationships between these things.
>
> (Jim Hendler)

Many of the current developments in ontology languages arise from interest in the *semantic web*, as briefly discussed in Chapter 1. The idea is to use ontologies to add information to web pages in such a way that it becomes possible for computers to process information on the page in meaningful and useful ways. For example, imagine we are searching on the Web to find out what the weather is like in Liverpool. Now, if our web browser 'knew' that 'Merseyside' is the same as 'Liverpool', and it found a web service providing the weather in Merseyside, then this would be enough to deduce that this was a useful service. The idea is that we can do this by using ontologies to annotate web services and the like with this kind of information.

In this chapter, we will look at the various ways in which researchers have gone about developing and deploying ontologies. Many of these techniques have their origins in the semantic web.

6.1 Ontology Fundamentals

For the most part, this chapter is concerned with presenting computer processable languages that can be used to define areas of common understanding between multiple agents. Before we go into the details of such languages, however, we will consider the basic concepts used to define ontologies, and how such ontologies might be used in practice.

6.1.1 Ontology building blocks

How do you start to define a common vocabulary? Well, in our everyday lives, we usually do this *by defining new terms with respect to old ones*. Consider the following fragment of a conversation:

> Alice: 'Did you read *Prey*?'
>
> Bob: 'No, what is it?'
>
> Alice: 'A science fiction novel. It's also a bit of a horror novel, actually. It's about multiagent systems going haywire.'

Let's unpack the information about *Prey* that is being conveyed here. We start with the

Let's unpack the information about *Prey* that is being conveyed here. We start with the obvious:

- *Prey* is a novel.
- *Prey* is a science fiction novel, i.e. belongs to the science fiction genre.
- *Prey* is a horror novel, i.e. belongs to the horror genre.
- The subject matter of *Prey* is multiagent systems going wrong.

It is important to realize that Alice is here defining *Prey* with respect to concepts that Bob is assumed to already understand. In particular, Alice is assuming that Bob knows what a novel is, what the science fiction genre is, and what the horror genre is. Thus Alice is defining a new term – *Prey* – with respect to concepts that Bob is already familiar with – novel, horror novel, and science fiction novel.

Notice that the entities in this discussion fall into two categories:

- *Classes* A class is a collection of things with similar properties. In the discussion above, the classes are 'novel', 'science fiction novel', and 'horror novel'.

CLASSES

- *Instances* (a.k.a. *objects*) An instance is a specific example of a class. In the discussion above, *Prey* is an instance of several classes: in particular, it is an instance of 'science fiction novel' and 'horror novel'.

Now, Bob probably has some *general* knowledge about novels, relating to the *properties* that novels typically have, and how they stand in relation to other things in the world.

PROPERTIES

For example, with respect to how novels stand in relation to other classes, Bob might know that *novels are works of fiction*, and so on. Here, 'work of fiction' is another class, and we are here saying that 'novel' is a *subclass* of 'work of fiction', or, equivalently, that 'work of fiction' is a superclass of 'novel'. If *A* is a subclass of *B*, then we often say that *B subsumes A*. With respect to the properties of novels, Bob might know that:

SUBCLASS SUPERCLASS

CLASS SUBSUMPTION

- novels have authors
- novels have publishers
- novels have publication dates
- novels contain many words
- … and so on.

Now, because Bob knows that *Prey* is a novel, he can assume that *Prey inherits the properties of novels*. Thus, he can assume that:

- *Prey* has an author
- *Prey* has a publication date

- *Prey* has a publication date
- *Prey* has a publisher
- … and so on.

But he can do more than this. The 'subclass' relation is inherently *transitive*: this means that if *A* is a subclass of *B*, and *B* is a subclass of *C*, then *A* is also a subclass of *C*. In other words, since *Prey* is a novel, and all novels are works of art, he can assume that *Prey* inherits the properties of works of art.

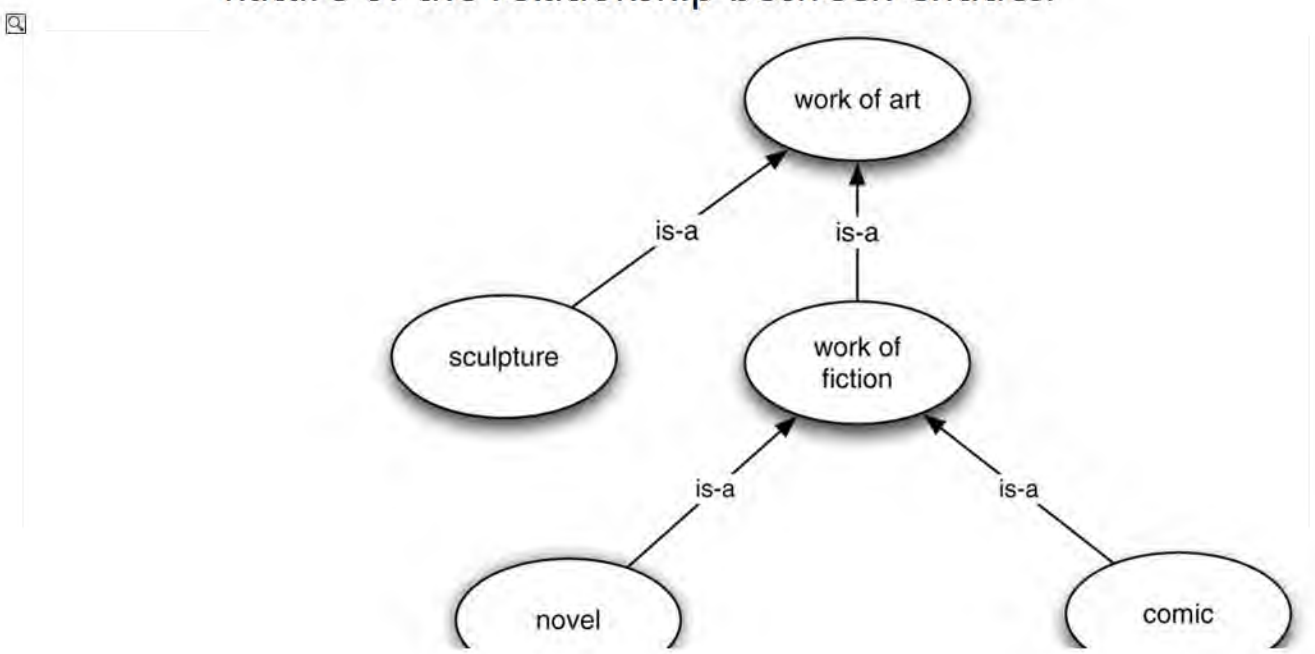
Of course, Bob knows lots of other things about the properties of novels and their relationship to other entities. For example, Bob might reasonably be expected to know that a work of fiction is a work of art (i.e. 'work of fiction' is a subclass of 'work of art') and also that 'sculpture' is also a subclass of 'work of art'. But he also knows that *no sculpture is a work of fiction*; i.e. the classes 'sculpture' and 'work of fiction' are *disjoint*. We typically want to capture these pieces of knowledge in an ontology, and we do this by writing what are called *axioms*.

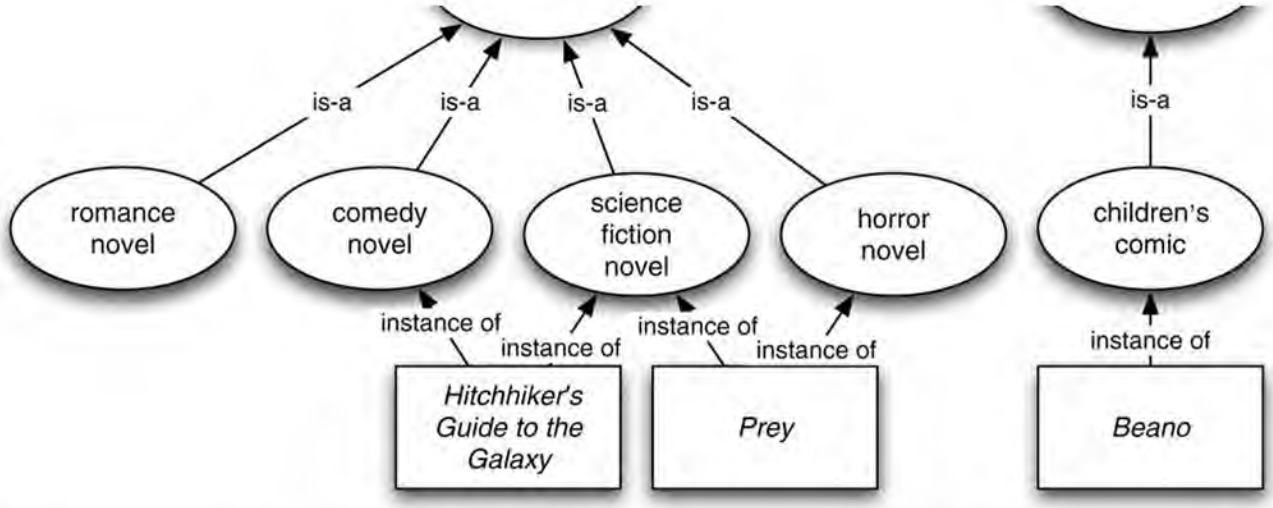
AXIOMS

If we were to draw a picture of Bob's state of knowledge after the conversation above, it might look something like Figure 6.1 (the term 'is-a' is usually used to indicate that one class '*is a*' subclass of another). In fact, Figure 6.1 defines an ontology, albeit a somewhat informal one. Ontologies typically use the taxonomic hierarchy evident in Figure 6.1, and typically make use of the idea of inheritance between classes. To be a bit more pernickety, however, the term 'ontology' is usually reserved for the *structural* part of Figure 6.1, i.e. the classes, their properties, and their interrelationships. An ontology together with a set of instances of classes defined in the ontology is called a *knowledge base*.

KNOWLEDGE BASE

Figure 6.1: A fragment of Bob's knowledge after a conversation about the novel *Prey*. Classes are drawn as ovals, and instances as rectangles. Labels on arrows indicate the nature of the relationship between entities.





6.1.2 An ontology of ontologies

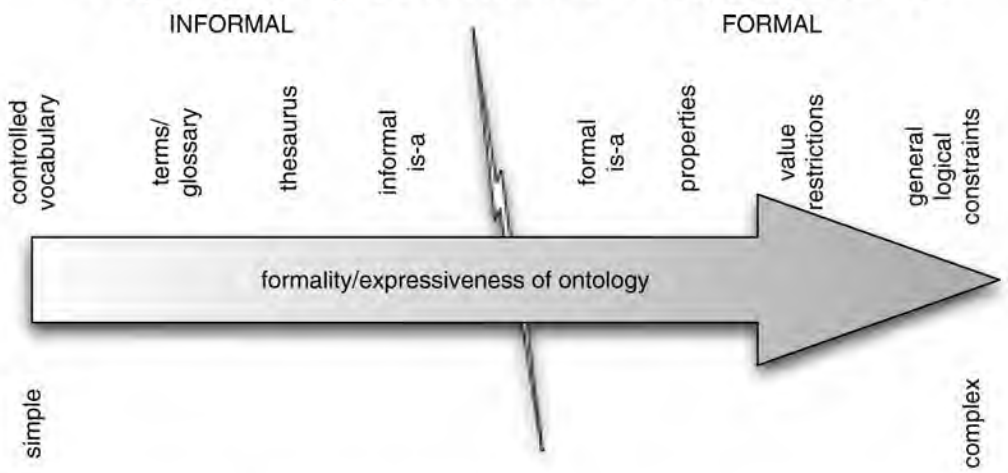
Depending on their intended use, ontologies come in a variety of types, varying in their formality and their specificity. We can think about the following informal classification of ontologies, depending on their formality [Lassila and McGuiness, 2001] (see Figure 6.2). We start with *informal ontologies*:

- *Controlled vocabulary* Perhaps the simplest kind of ontology is a *controlled vocabulary*. In a controlled vocabulary, we make use of a few predefined keywords to classify entities: no hierarchies, properties, or axioms. In some domains, this is sufficient.

  CONTROLLED VOCABULARY

- *Terms/glossary* Here, we have a list of terms, as with a controlled vocabulary, but some attempt is made (typically with natural language, e.g. an English explanation) to define the meaning of these terms. However, the computational power of such an ontology is roughly that of a controlled vocabulary, since we cannot in general compute with the natural language explanation. The value of the explanation is therefore usually for the ontology designer.

Figure 6.2: The ontology spectrum: from informal and less expressive up to formal and very expressive.



- *Thesaurus* A thesaurus defines *synonyms*: terms that have the same meaning. Thus, to pick a silly example, if you want to find a web service that provides weather forecasts, it might be useful to know that ‘meteorological forecast’ means the same thing.

- *Informal 'is-a' taxonomies* Here we think of controlled vocabularies organized into an informal hierarchy. We find such hierarchies on web sites such as Amazon.com, for example, where goods for sale are organized into loose hierarchies. Typically the hierarchies are not formal hierarchies, because related goods (e.g. cameras and camera bags) are collected together in the same place, without any formal definition of how or why the goods are clustered in this way.

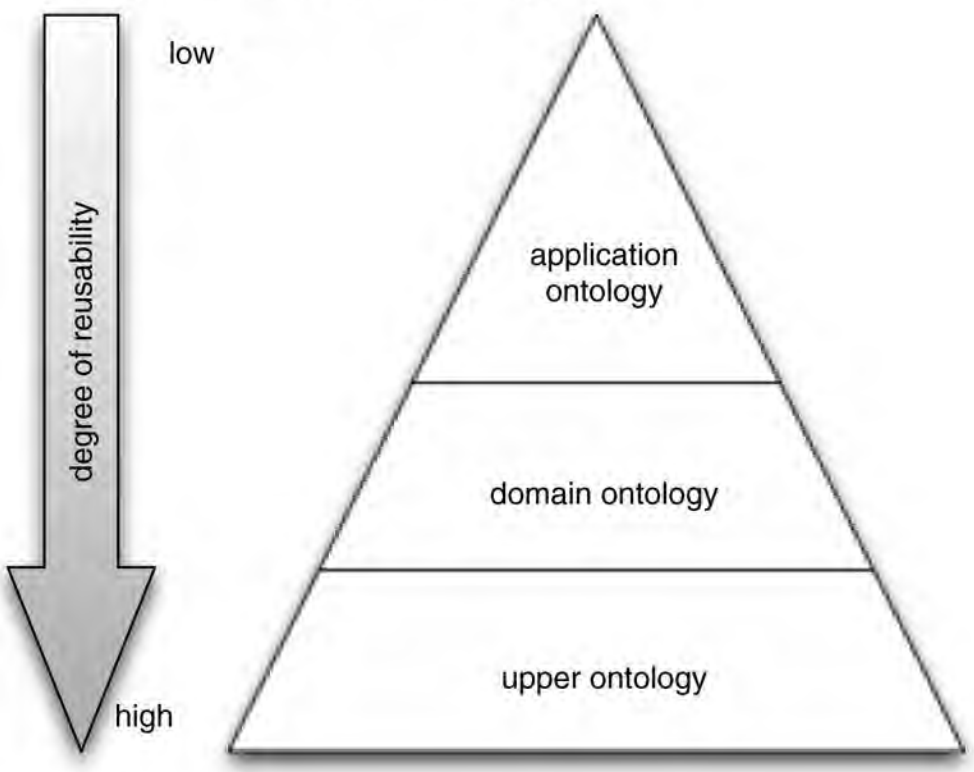
Next, we move to formal ontologies, where some attempt is made to give the terms in the ontology some formal semantics.

- *Formal 'is-a' taxonomies* Here, we explicitly define subsumption relationships between classes.
- *Properties* Here, we now allow classes to have properties, and together with the subsumption relation, this permits us to draw conclusions about the properties of classes.
- *Value restrictions* Value restrictions give additional information about relationships; for example, a typical restriction might say that 'every person has exactly one birth mother'.

VALUE RESTRICTIONS

- *Arbitrary logical constraints* Finally, we might have ontologies with arbitrary logical constraints. Such constraints go beyond value restrictions, taxonomical hierarchies, etc. In general, such constraints allow us a great degree of precision when defining an ontology. The drawback with such constraints is that, in general, allowing arbitrary logical expressions leads to very high computational complexity (and even undecidability) with respect to reasoning.

Figure 6.3: The ontology hierarchy: from very general (and reusable) at the bottom, to very specific (and not very reusable) at the top.



As well as distinguishing ontologies based on their formality and the types of information that

they convey, we can also usefully distinguish ontologies based on their role in an application: see Figure 6.3. At the bottom, we see the most general kind of ontology: the so-called 'upper' ontology (even though it appears at the bottom in Figure 6.3!) Such an ontology might start out by defining the most general class imaginable ('thing') and then define classes that specialize this (e.g. 'living thing', 'non-living thing').

UPPER ONTOLOGY

DOMAIN ONTOLOGY

A *domain ontology* defines concepts appropriate for a specific application domain. For example, it might define concepts relating to medical terminology, and be used by a number of applications in the area of medicine [Obofoundry, 2008]. Note that a domain ontology will typically build upon and make use of concepts from an upper ontology: this idea of *reuse* of ontologies is very important, as the more applications use a particular ontology, the more agreement there will be on terms.

APPLICATION ONTOLOGY

Finally, an *application ontology* defines concepts used by a specific application. Again, it will typically build upon a domain ontology and in turn upon some upper ontology. Concepts from an application ontology will not usually be reusable: they will typically be of relevance only within the application for which they were defined.

6.2 Ontology Languages

The ideas presented above can be realized in many different ways, and over the years, a bewildering range of ontology languages have been proposed. In this section, we will survey the most popular approaches, starting with the least formal.

6.2.1 XML – ad hoc ontologies

AD HOC ONTOLOGIES

Arguably the simplest ontologies to create and use are *ad hoc* ontologies: created with little effort, for a specific purpose, usually with a short expected period of use. Often such ontologies take the form of a controlled vocabulary, and *XML* is usually the language of choice for such ontologies.

The extensible markup language (XML) [XML, 2008] is not an ontology language, although it can be directly used to define simple ontologies in an informal way. It is best thought of as a kind of extension to HTML, which in a nutshell allows us to define our own tags and document structures. To understand how XML came about, it is necessary to look at the history of the Web. The Web essentially comprises two things: a protocol (HTTP), which provides a common set of rules for enabling web servers and clients to communicate with one another, and a format for documents called (as I am sure you know!) the hypertext markup language (HTML). Now HTML essentially defines a grammar for interspersing documents with *markup commands*. Most of these markup commands relate to document layout, and thus give indications to a web browser of how to display a document: which parts of the document should be treated as section headers, emphasized text, and so on. Of course, markup is not

restricted to layout information: programs, for example in the form of JavaScript code, can also be attached. The grammar of HTML is defined by a *document type declaration* (DTD). A DTD can be thought of as being analogous to the formal grammars used to define the syntax of programming languages. The HTML DTD thus defines what constitutes a syntactically acceptable HTML document. A DTD is in fact itself expressed in a formal language – the standard generalized markup language [SGML, 2001]. SGML is essentially a language for defining other languages.

XML

MARKUP

DTD

SGML

Now, to all intents and purposes, the HTML standard is fixed, in the sense that you cannot arbitrarily introduce tags and attributes into HTML documents that were not defined in the HTML DTD. But this severely limits the usefulness of the Web. To see what I mean by this, consider the following example. An e-commerce company selling CDs wishes to put details of its prices on its web page. Using conventional HTML techniques, a web page designer can only mark up the document with layout information (see, for example, Figure 6.4(a)). But this means that a web browser – or indeed any program that looks at documents on the Web – has no way of knowing which parts of the document refer to the titles of CDs, which refer to their prices, and so on.

Now, using XML it is possible to define *new* markup tags – and so, in essence, to extend HTML. To see the value of this, consider Figure 6.4(b), which shows the same information as Figure 6.4a, expressed using new tags (catalogue, product, and so on) that were defined using XML. Note that new tags such as these cannot be arbitrarily introduced into HTML documents: they must be defined. The way they are defined is by writing an XML DTD; thus XML, like SGML, is a language for defining languages. (In fact, XML is a subset of SGML.)

Figure 6.4: Plain HTML versus XML.

(a) Plain HTML

```
<ul>
   <li><em>Music</em>,
      <b>Madonna<b>,
      USD12<br><p>
   <li><em>Get Ready</em>,
      <b>New Order</b>,
      USD14<br><p>
</ul>
```

(b) XML

```
<catalogue>
   <product type="CD">
      <title>Music</title>
      <artist>Madonna</artist>
```

```
      <price currency="USD">12</price>
   </product>
   <product type="CD">
      <title>Get Ready</title>
      <artist>New Order</artist>
      <price currency="USD">14</price>
   </product>
</catalogue>
```

I hope it is clear that a computer program would have a much easier time *understanding the meaning* of Figure 6.4b than it would with Figure 6.4a. In Figure 6.4a, there is nothing to help a program understand which part of the document refers to the price of the product, which refers to the title of the product, and so on. In contrast, Figure 6.4b makes all this explicit. We can think of the tags in Figure 6.4b as being a controlled vocabulary, providing a useful but relatively informal ontology.

6.2.2 OWL – The web ontology language

OWL is probably the most important and influential ontology language at the time of writing [Bechhofer et al., 2004]. To be a bit more precise, OWL is a collection of several XML-based ontology frameworks, within which ontologies in these various frameworks can be expressed. Specifically, there are three main 'levels' of OWL, as follows:

OWL LITE

- *OWL Lite*. This is the simplest (least expressive) variant of OWL, which supports only basic ontology features. In particular, OWL Lite places a number of restrictions on the types of axioms one can write. The point about these restrictions is that they result in a language that is computationally more tractable (and is also somewhat easier for humans to use and understand) than more expressive OWL variants.
- *OWL DL*. This language extends the properties of OWL Lite; for example, it permits one to express the fact that two classes are disjoint. The features of OWL DL were carefully chosen so that the language corresponds exactly to a particular formalism known as *description logic* [Baader et al., 2003].

  OWL DL

  DESCRIPTION LOGIC
- *OWL Full*. This is a very expressive framework, providing many features for defining ontologies; however, in its full glory, the framework is so rich that many reasoning problems with OWL Full (such as consistency checking) are undecidable.

  OWL FULL

Notice that the same ontology can be expressed in many different languages. This may seem a strange idea at first: the following metaphor may help. Think about an ontology as being like an algorithm; an algorithm exists independently of a programming language, and can be concretely expressed in any number of different programming languages. In just the same way, we can write down an OWL ontology in a number of different languages. Perhaps the

most important distinction is between OWL's *concrete syntax* and *abstract syntax*. The concrete syntax is intended primarily for computers (i.e. it is not really intended for human readers), and is based on XML. A fragment of an OWL knowledge base, expressed using the OWL concrete syntax (from the OWL version of the CIA world fact book) is shown in Figure 6.5.

CONCRETE/ABSTRACT SYNTAX

This concrete language is in fact hard for people to read, and rather verbose. The *abstract syntax* is intended to be more user-friendly. It provides a concise and readable humanoriented language for defining OWL ontologies; this language can easily be automatically translated to the concrete XML syntax. For these reasons, we will here focus on the abstract syntax.

I will start with a simple example of an OWL representation of Figure 6.1 – see Figure 6.6 (refer to Table 6.1 for a list of the main OWL constructs). Lines 2–11 introduce the classes in the ontology. All of these definitions basically say that we define a class by saying it is a subclass of another. For example, the line

```
Class(WorkOfArt partial owl:Thing)
```

introduces a new class, `WorkOfArt`, and says that this class is a subclass of the class `owl:Thing`. The class `owl:Thing` is a predefined OWL class, intended to be the most general class – the set of everything. (Another predefined class, which we don't use here, is owl:Nothing, which is the empty set.) The keyword '`partial`' here means that this line is a *partial definition of the class*, in the sense that it is not a *complete* definition: the conditions given are *necessary*, but not *sufficient*.

The keyword complete can be used to indicate that the definition given is complete, i.e. the class is defined by a set of conditions that are both necessary and sufficient. Here is a (rather silly) example of a complete class definition, which makes use of the oneOf construct in Table 6.1.

```
Class(Vowel complete oneOf ("a" "e" "i" "o" "u"))
```

Figure 6.5: Some facts about the UK, expressed in OWL.

```
<NS1:geographicCoordinates rdf:nodeID='A6'/>
<NS1:mapReferences>Europe</NS1:mapReferences>
<NS1:totalArea>244820</NS1:totalArea>
<NS1:waterArea>3230</NS1:waterArea>
<NS1:landArea>241590</NS1:landArea>
<NS1:comparativeArea>
   slightly smaller than Oregon
</NS1:comparativeArea>
<NS1:landBoundaries>360</NS1:landBoundaries>
<NS1:coastline>12429</NS1:coastline>
<NS1:exclusiveFishingZone>200</NS1:exclusiveFishingZone>
<NS1:territorialSea>12</NS1:territorialSea>
<NS1:climate>
   temperate; moderated by prevailing
   southwest winds over the North Atlantic
   Current; more than one-half of the days
   are overcast
</NS1:climate>
<NS1:terrain>
```

```
        mostly rugged hills and low mountains;
        level to rolling plains in east and
        southeast
    </NS1:terrain>
```

Thus the expression `oneOf("a" "e" "i" "o" "u")` defines a class containing the vowels (i.e. the letters 'a', 'e', 'i', 'o', 'u'), and the class Vowel is defined to be exactly this class. In stating that a class definition is `partial`, as opposed to `complete`, we are basically saying that there is more to say about the class – some other constraints and properties might come later.

Line 12 in Figure 6.6 defines an axiom: it says that the classes `Sculpture` and `WorkOfFiction` are disjoint, i.e. that these classes have no objects in common.

Lines 13–14 and 15–16 define two properties: for example, the declaration

```
ObjectProperty(author
 domain(Novel) range(Person))
```

defines a property author, and says that the domain of this property is the class `Novel` (i.e. every object in the class `Novel` has this property), and the range of the property is the class `Person`; that is, the `author` of a `Novel` is a `Person`. (Note that we haven't in fact defined the classes `Person` or `String`; in fact, with OWL one can make use of the basic XML datatypes.)

Lines 17–22, 23–27, and 28–29 define three individuals, i.e. three objects. For example, lines 17–22 define an object 'Hitchhiker's Guide to the Galaxy'.

```
Individual("Hitchhiker's Guide to the Galaxy"
type(ScienceFictionNovel)
```

Figure 6.6: A simple ontology in the OWL abstract syntax notation (line numbers are for reference only, and are not part of the specification).

```
1.  Ontology(
2.     Class(WorkOfArt partial owl:Thing)
3.     Class(Sculpture partial WorkOfArt)
4.     Class(WorkOfFiction partial WorkOfArt)
5.     Class(Novel partial WorkOfFiction)
6.     Class(Comic partial WorkOfFiction)
7.     Class(RomanceNovel partial Novel)
8.     Class(ComedyNovel partial Novel)
9.     Class(ScienceFictionNovel partial Novel)
10.    Class(HorrorNovel partial Novel)
11.    Class(ChildrensComic partial Comic)
12.    DisjointClasses(Sculpture WorkOfFiction)
13.    ObjectProperty(author
14.       domain(Novel) range(Person))
15.    ObjectProperty(content
16.       domain(Novel) range(String))
17.    Individual("Hitchhiker's Guide to the Galaxy"
18.       type(ScienceFictionNovel)
19.       value(author "Douglas Adams")
```

```
20.       value(content "Far out in the uncharted
21.       backwaters of the unfashionable end of
22.       the Western Spiral Arm of the Galaxy..."))
23.    Individual("Prey"
24.       type(intersectionOf(HorrorNovel ScienceFictionNovel))
25.       value(author "Michael Crichton")
26.       value(content "Things never turn out
27.          the way you think they will...."))
28.    Individual("Beano"
29.       type(ChildrensComic))
30.    DifferentIndividuals(
31.       "Hitchhiker's Guide to the Galaxy"
32.       "Prey")
33.)
```

```
value(author "Douglas Adams")
value(content "The Hitchhiker's Guide
to the Galaxy is a wholly remarkable
book, perhaps the most remarkable ..."))
```

This definition states that the `type` of this object is `ScienceFictionNovel` (i.e. 'Hitchhiker's Guide to the Galaxy' is an instance of the class `ScienceFictionNovel`), and gives `values` for the properties `author` and `content` (I didn't give the complete value for `content`!).

Table 6.1: OWL constructs.

Class constructs	
`intersectionOf(A B)`	set-theoretic intersection of classes `A` and B, i.e. `A` ∩ `B`
`unionOf(A B)`	set-theoretic union of classes `A` and `B`, i.e. `A` ∪ `B`
`complementOf(A)`	set-theoretic complement of class `A`
`oneOf(o1 o2 … on)`	defines a class by listing all its objects `o1` to `on`
Property constructs	
`allValuesFrom(A)`	universal quantification for properties (see text)
`someValuesFrom(A)`	existential quantification for properties (see text)
`hasValue(e)`	the relevant property takes value *e*
`minCardinality(n)`	the relevant property has cardinality at least *n*
`maxCardinality(n)`	the relevant property has cardinality at most *n*
`inverseOf`	the inverse of the property
Restrictions for classes and properties	
`SubClassOf(A B)`	`An` 'is-a' subclass of `B`
`EquivalentClasses(A B)`	`A` and `B` are equivalent
`SubPropertyOf(p1 p2)`	`p1` is a subproperty of `p2`
`EquivalentProperties(p1 p2)`	properties `p1` and `p2` are equivalent
`SameIndividual(o1 o2)`	individuals denoted by `o1` and `o2` are the same

`DisjointClasses(A B)`	no object is an instance of both `A` and `B`
`DifferentIndividuals(o1 o2)`	objects denoted by `o1` and `o2` are different
`InverseOf(P1 P2)`	property `P1` is the inverse of `P2`
`Transitive(P)`	property `P` is transitive

The definition of the object 'Prey' is similar, but here note that a slightly more complex `type` definition is given:

```
type(intersectionOf(HorrorNovel ScienceFictionNovel))
```

The `intsersectionOf(...)` operator here defines a new type that is the intersection of `HorrorNovel` and `ScienceFictionNovel`. We can build up complex type definitions in this way using the other class constructs in Table 6.1 (`unionOf, complementOf, ...`).

Finally, lines 30–32 say that the two novels introduced earlier are different (this may seem strange, but OWL does not exclude as a logical possibility that they are the same individual unless explicitly told to).

Before leaving OWL, we will say a few words about the other operators in Table 6.1 that are not used in the example. First, OWL allows us to use *quantification* with respect to properties. For example, suppose we wanted to define a new class, `HorrorWriters`, for authors who *only* write horror novels, (i.e. a horror writer is somebody who never writes any other kind of fiction, such as science fiction). First, since the `author` property applies to the class `Novel`, we will introduce the *inverse property* of author:

INVERSE PROPERTY

```
ObjectProperty(authorOf inverseOf(author))
```

Thus `authorOf` is a property with domain `Person` and range `Novel`.

Next, we want to define the class of horror authors to be people all of whose books are horror novels. The key to doing this is to use the `allValuesFrom` restriction. We first present a version that almost works, but isn't quite right.

```
Class(HorrorAuthor
 complete Person
 restriction(authorOf allValuesFrom(HorrorNovel)))
```

The definition says that a `HorrorAuthor` is completely defined as a person which satisfies a particular `restriction` on its properties, in particular, that all the values from the `authorOf` property must belong to the class `HorrorNovel` (i.e. all the books that the person has written must be horror books). The reason that this definition doesn't quite work is as follows: consider somebody who hasn't written any books at all. In this case, by definition, everything they *have* written is a horror novel. (This situation should be familiar to readers with some understanding of first-order logic: `allValuesFrom` is behaving like a universal quantifier.)

So, we need to refine the definition a bit: basically, we need to say that a `HorrorAuthor` is a person all of whose books are `HorrorNovels`, and, in addition, who *has written at least one novel*. The following serves as an appropriate definition.

```
Class(HorrorAuthor
 complete intersectionOf(
```

```
  restriction(authorOf allValuesFrom(HorrorNovel)) Person
  restriction(authorOf minCardinality(1))))
```

The associated `someValuesFrom` operator allows us to use *existential quantification*. For example, we can define an `OccassionalHorrorAuthor` as somebody who has written *at least one book:* the following OWL definition would achieve this.

```
Class(OccassionalHorrorAuthor
 complete Person
 restriction(authorOf someValuesFrom(HorrorNovel)))
```

In OWL, the general knowledge about classes – the class hierarchies and the axioms that define properties of relationships – is often called a *TBox* (for 'terminological box'), while knowledge about instances is called an *ABox* ('assertion box').

TBOX

ABOX

Reasoning with OWL

Suppose we have an OWL ontology, such as that presented in Figure 6.6. There are several reasoning tasks associated with such ontologies, some relating to the *design* of the ontology, and some relating to its *use*. In addition, one can classify reasoning tasks depending on whether we are dealing with just classes and their relationships (the TBox) or assertions as well (TBox and ABox). With respect to reasoning about classes, we have the following.

CONSISTENCY CHECKING

- *Consistency checking* This is a reasoning task that will be of interest primarily to the ontology designer. The consistency checking problem is simply that of telling whether an ontology contains any explicit or implicit contradictions. Such a contradiction might be along the lines of '*A* is a subclass of *B, B* is a subclass of *C, C* is not a subclass of *A*'. The transitivity of the subclass relationship tells us that there is no way that this last property can be true if the first two subclass relationships hold: the ontology is inherently contradictory. Often, when defining an ontology with complex class hierarchies and associated axioms, it is hard to spot inconsistencies (they may be much more subtle than the example I have given here). It is therefore important to be able to tell whether an ontology is consistent in this sense.
- *Concept satisfiability* A slightly more limited notion of consistency is the following: we are given an ontology *O* and a class *A* appearing in *O*. The question is whether *A* is non-empty, i.e. whether it can in principle have any members. For example, suppose you define a knowledge base, in which you say that classes *A* and *B* are disjoint, and then define a class *C* to be the intersection of *A* and *B*. Clearly, *C* cannot have any members, and usually this indicates a bug in the ontology. Again, this is a task that will be of relevance primarily for the ontology designer.

  CONCEPT SATISFIABILITY

- *Computing the subsumption hierarchy* This problem involves determining all the

- *Computing the subsumption hierarchy* This problem involves determining all the subsumption relations between classes in the ontology, i.e. for every pair of classes A and B, determine whether A is a subclass of B, vice versa, both, or neither. In other words, we compute the entire subsumption hierarchy.
- *Class subsumption* Class subsumption is similar to the above problem. We are given two classes A and B and simply asked whether A is a subclass of B.

CLASS SUBSUMPTION

- *Least common subsumer* Here, we are given a collection of classes, $A_1, \ldots, A_k$, and asked to find the most specific class that contains them all. For example, in Figure 6.1, the least common subsumer of 'comedy novel' and 'children's comic' is 'work of fiction', while the least common subsumer of 'children's comic' and 'sculpture' is 'work of art'.

LEAST COMMON SUBSUMER

If we consider reasoning associated with both concepts and instances, we have an additional reasoning problem: *instance classification*. Suppose you have an ontology O, and you are given a new object e. You know something about the properties of e, but don't know exactly where in your ontology this thing fits. For example, you might be presented with something that is a work of art, that contains many words, and that has an author. What you want would typically be the most specific classification of e that you can obtain using the knowledge in O. (The most specific classification will be the one that gives you the most information about the object.) In this case, you might conclude that e is a novel.

INSTANCE CLASSIFICATION

6.2.3 KIF – ontologies in first-order logic

I conclude by describing the *knowledge interchange format* – KIF [Genesereth and Fikes, 1992]. KIF is closely based on first-order logic [Enderton, 1972; Genesereth and Nilsson, 1987]. (In fact, KIF looks very like first-order logic recast in a LISP-like notation; to fully understand the details of this section, some understanding of first-order logic is therefore helpful.) Thus, for example, by using KIF, it is possible for agents to express:

KIF

- properties of things in a domain (e.g. 'Michael is a vegetarian' – Michael has the property of being a vegetarian)
- relationships between things in a domain (e.g. 'Michael and Janine are married' – the relationship of marriage exists between Michael and Janine)
- general properties of a domain (e.g. 'everybody has a mother').

In order to express these things, KIF assumes a basic, fixed logical apparatus, which contains the usual connectives that one finds in first-order logic: the binary Boolean connectives `and`, `or`, `not`, and so on, and the universal and existential quantifiers `forall` and `exists`. In addition, KIF provides a basic vocabulary of objects – in particular, numbers, characters, and strings. Some standard functions and relations for these objects are also